

CKA: Core Concepts

KodeKloud Complete Study Notes — Certified Kubernetes
Administrator

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Core Concepts Section Introduction

This article introduces essential concepts of modern Cluster Architecture, covering key API primitives for managing containerized applications.

This article introduces the essential concepts underpinning modern Cluster Architecture. We start with a high-level overview to help you understand how various components form a cohesive structure. Following the overview, the discussion advances to key API primitives—including Pods, ReplicaSets, Deployments, and Services—that are integral to managing containerized applications.

If you already have foundational training or have completed certification-level learning for Application Developers, you may opt to bypass some sections and focus solely on the practice tests.

Let's embark on our exploration of Cluster Architecture and uncover how these components interact to deliver a robust deployment environment.

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Cluster Architecture

This article provides an overview of Kubernetes cluster architecture, detailing the roles and components of master and worker nodes in managing containerized applications.

Hello and welcome to our comprehensive guide on Kubernetes cluster architecture. In this article, we provide a high-level overview of how Kubernetes organizes and manages containerized applications. You will learn about each component's roles, responsibilities, and configurations, as well as practical insights into examining an existing cluster.

Kubernetes simplifies the deployment, scaling, and management of containerized applications through automation. To help explain this concept, imagine two kinds of ships: cargo ships (worker nodes) that carry containers, and control ships (master nodes) that monitor and manage the cargo ships. In Kubernetes, the cluster consists of nodes—whether physical or virtual, on-premises or cloud-hosted—that host your containerized applications.

Master Node Components

The master node contains several control plane components that manage the entire Kubernetes cluster. It keeps track of all nodes, decides where applications should run, and continuously monitors the cluster. Think of the master node as the central command center coordinating the fleet.



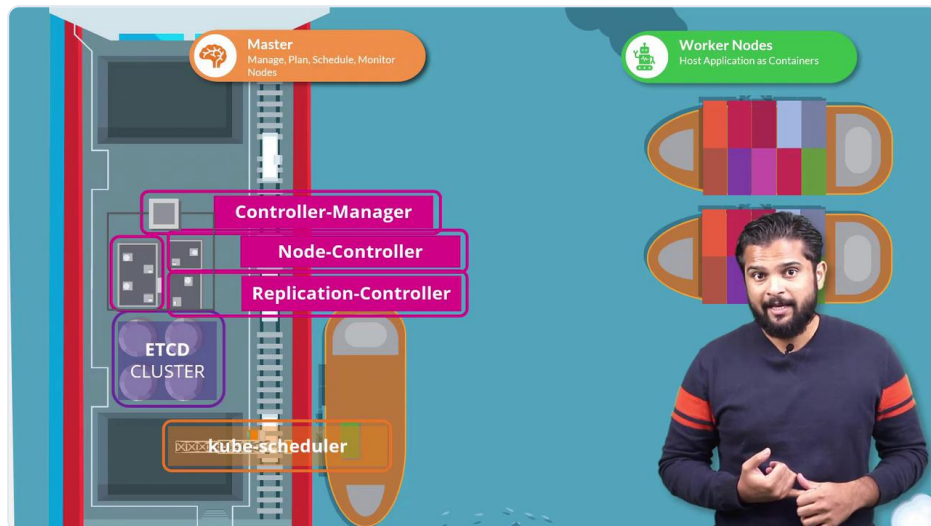
In a busy harbor, many containers are loaded and unloaded daily. Kubernetes maintains detailed information about each container and its corresponding node in a highly available key-value store called etcd. Etcd uses a simple key-value format along with a quorum mechanism, ensuring reliable and consistent data storage across the cluster.

When a new container (or "ship cargo") is ready, the Kubernetes scheduler—similar to port cranes—determines which worker node (or "ship") should host it. The scheduler takes into account current load, resource requirements, and specific constraints like taints, tolerations, or node affinity rules. This scheduling process is vital for efficient cluster operation.

The Kubernetes replication controller and other controllers work like dock office staff, ensuring that the desired number of containers are running and managing node operations.

Other key master node components include:

- **ETCD Cluster:** Stores cluster-wide configuration and state data.
- **Kube Scheduler:** Determines the best node for new container deployments.
- **Controllers:** Manage node lifecycle, container replication, and system stability.
- **Kube API Server:** Acts as the central hub for cluster communication and management.



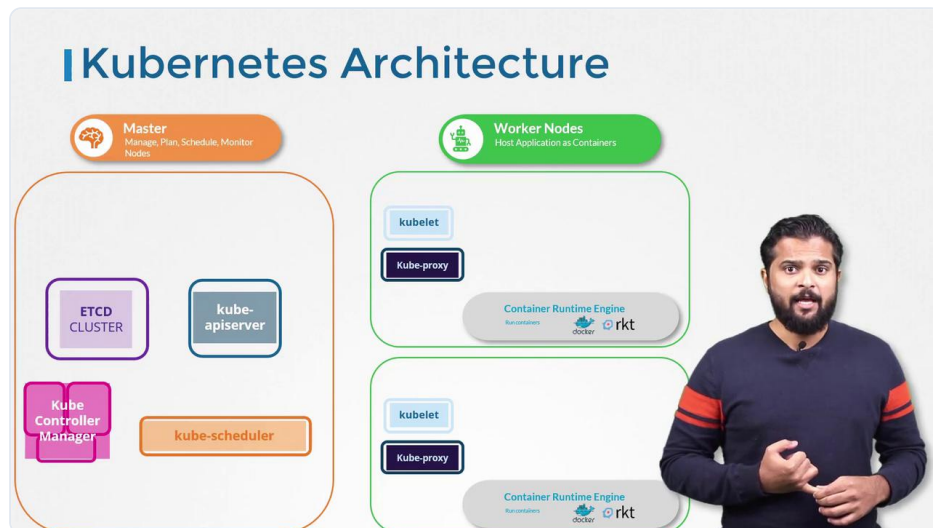
Worker Node Components

Worker nodes, which can be compared to cargo ships, are responsible for running the containerized applications. Each node is managed by the Kubelet, the node's "captain," which ensures that containers are running as instructed.

- **Kubelet:** Manages container lifecycle on an individual node. It receives instructions from the Kube API server to create, update, or delete containers, and regularly reports the node's status.
- **Kube Proxy:** Configures networking rules on worker nodes, thus enabling smooth inter-container communication across nodes. For instance, it allows a web server on one node to interact with a database on another.

The entire control system is containerized. Whether you are using Docker, Containerd, or CRI-O, every node (including master nodes with containerized components) requires a compatible container runtime engine.

The high-level worker node architecture ensures that applications remain available and responsive, even as they communicate across a distributed network.



Summary of Kubernetes Architecture

The Kubernetes cluster architecture is divided into two main segments:

Component Category	Key Components	Description
Master Node	etcd, Kube Scheduler, Controllers, Kube API Server	Centralized control and management of the entire cluster.
Worker Node	Kubelet, Kube Proxy	Responsible for the lifecycle management of containers and ensuring network communication between services.

This clear separation and coordination between master and worker nodes is fundamental to Kubernetes' ability to automate and streamline container orchestration.

We hope this detailed overview of Kubernetes cluster architecture has provided valuable insights. In upcoming articles, we will explore each component in depth, offering practical examples and exercises to further enhance your understanding of Kubernetes systems.

Happy learning and stay tuned for more Kubernetes content!

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Docker vs ContainerD

This article explores the relationship between Docker and ContainerD, detailing their differences and the CLI tools for interacting with container runtimes.

In this lesson, we explore the intricate relationship between Docker and ContainerD, as well as the various CLI tools that interact with container runtimes. If you've encountered references to Docker alongside Kubernetes or ContainerD in different articles, this guide will clarify their differences and explain when to use tools like CTR, CRICTL, or NodeController.

The Evolution of Container Runtimes

At the dawn of the container era, Docker reigned supreme thanks to its intuitive interface. Even though alternative tools like Rocket existed, Kubernetes was initially designed to orchestrate Docker-based containers, creating a strong coupling between Docker and Kubernetes. However, as Kubernetes expanded its ecosystem, other container runtimes sought integration.

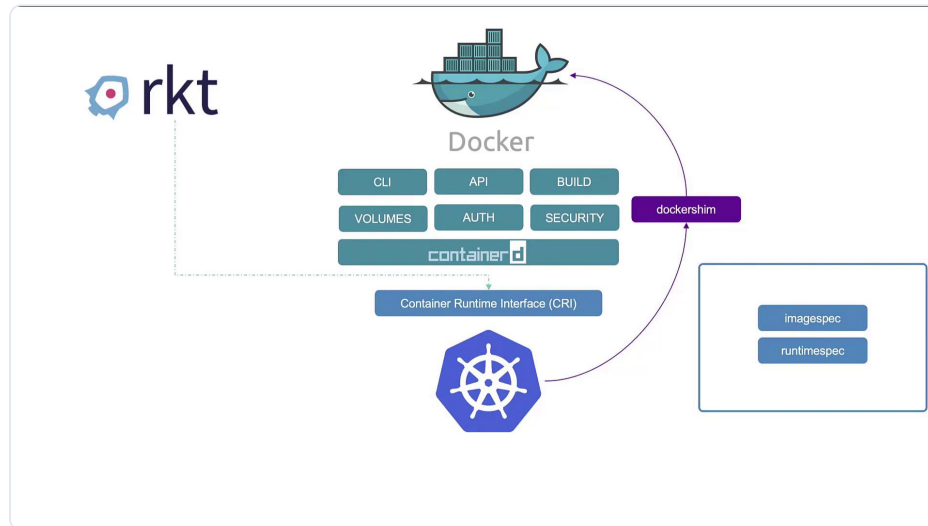
Kubernetes addressed this need by introducing the Container Runtime Interface (CRI). The CRI standardizes container runtimes by ensuring compliance with the Open Container Initiative (OCI) standards—the image specification (for building images) and the runtime specification (for container execution). This standardization paved the way for runtimes like Rocket and others to be supported by Kubernetes.

Since Docker was developed before CRI was established, it wasn't built with CRI compatibility in mind. To allow Docker to function with Kubernetes, a temporary solution known as the Docker Shim was implemented. Over time, native CRI-compatible runtimes such as ContainerD emerged, shifting Docker's primary role within the Kubernetes architecture.

ContainerD, a CRI-compatible runtime, integrates directly with Kubernetes—eliminating the need for the Docker Shim. Originally bundled with Docker, ContainerD has evolved into a standalone project under the Cloud Native

Computing Foundation. This decoupling means you can install and use ContainerD without needing the entire Docker ecosystem if your goal is solely container runtime functionality.

The diagram below illustrates the relationship between Docker, Kubernetes, and alternative container runtimes (like rkt), highlighting components such as containerd, the Docker Shim, and the Container Runtime Interface (CRI).



Within Docker, several components work together to provide its functionality:

- Docker CLI and API
- Image build tools
- Support for volumes, authentication, and security
- The container runtime (runc) managed by containerd

Since ContainerD is CRI-compatible, it works directly with Kubernetes, which eventually led to Kubernetes version 1.24 removing support for Docker as a runtime due to the complexity of maintaining the Docker Shim. However, Docker images remain fully OCI-compliant and continue to work seamlessly with ContainerD.

Diving Deeper into ContainerD

ContainerD began as an internal component of Docker but is now maintained as an independent project. If you do not require Docker's additional features, you can install ContainerD on its own. Traditionally, Docker users ran containers using commands like `docker run`, but ContainerD introduces different CLI tools.

Note: Installation Example To install ContainerD, execute the following command:

```
```bash theme={null} $ tar -C /usr/local -zxvf containerd-1.6.2-linux-  
amd64.tar.gz bin/ bin/ctr bin/containerd ...
```

Once installed, CTR can pull images and execute basic container operations. However, due to

<Frame>

! [The image describes the "ctr" CLI tool, bundled with containerD as not user-friendly

</Frame>

### Using CTR

For example, to pull and run a Redis image using CTR, use the following commands:

```
```bash theme={null}  
$ ctr images pull docker.io/library/redis:alpine  
$ ctr run docker.io/library/redis:alpine redis
```

NerdCTL: A Docker-like CLI for ContainerD

NerdCTL offers a CLI experience similar to Docker's while providing additional features unique to ContainerD. Key features include:

- Encrypted container images
- Lazy pulling of images
- Peer-to-peer image distribution
- Image signing and verification in Kubernetes namespaces

By transitioning from Docker commands to NerdCTL, you maintain familiar syntax. For example, you can replace these Docker commands:

```
```bash theme={null} $ docker run --name redis redis:alpine $ docker run --  
name webserver -p 80:80 -d nginx
```

with the equivalent NerdCTL commands by simply substituting "docker" with "nerdctl" in your

## ## CRI Tools: A Kubernetes Perspective

Another significant CLI utility is CRI CTL (cricctl), designed to interact with any CRI-compliant

<Frame>

! [The image explains "cricctl," a CLI for CRI-compatible container runtimes, used for in

</Frame>

Cricctl is mainly used for tasks such as:

- \* Pulling container images
- \* Listing images and containers
- \* Inspecting logs and executing commands inside containers (using options like `-i` and `-t`)
- \* Listing pods (a functionality not provided by Docker commands)

For example, typical cricctl usage might include:

```
```bash
theme={null}
$ cricctl pull busybox
$ cricctl images
$ cricctl ps -a
```

It is important to note that containers manually created using cricctl may be removed by the Kubelet, as they are not registered as part of a Kubernetes Pod.

A comparison between the Docker CLI and cricctl shows many overlapping commands. Both tools support functions such as attach, exec, images, info, inspect, logs, ps, stats, and version, though there are differences in how they handle container creation and management.

docker vs cricctl

Retrieve debugging information ⓘ

docker cli	crictl	Description	Unsupported Features
attach	attach	Attach to a running container	<code>--detach-keys</code> , <code>--sig-proxy</code>
exec	exec	Run a command in a running container	<code>--privileged</code> , <code>--user</code> , <code>--detach-keys</code>
images	images	List images	
info	info	Display system-wide information	
inspect	inspect , inspecti	Return low-level information on a container, image or task	
logs	logs	Fetch the logs of a container	<code>--details</code>
ps	ps	List containers	
stats	stats	Display a live stream of container(s) resource usage statistics	Column: NET/BLOCK I/O, PIDs
version	version	Show the runtime (Docker, ContainerD, or others) version information	

<https://kubernetes.io/docs/reference/tools/map-crictl-dockercli/>

docker vs crictl

Perform Changes

docker cli	crictl	Description	Unsupported Features
create	create	Create a new container	
kill	stop (timeout = 0)	Kill one or more running container	--signal
pull	pull	Pull an image or a repository from a registry	--all-tags, --disable-content-trust
rm	rm	Remove one or more containers	
rmi	rmi	Remove one or more images	
run	run	Run a command in a new container	
start	start	Start one or more stopped containers	--detach-keys
stop	stop	Stop one or more running containers	
update	update	Update configuration of one or more containers	--restart, --blkio-weight and some other resource limit not supported by CRI.

<https://kubernetes.io/docs/reference/tools/map-crictl-dockercli/>

Changes in Kubernetes Runtime Endpoints

Earlier versions of Kubernetes had the kubelet connect to container runtime endpoints in the following default order:

```
```text theme={null} unix:///var/run/dockershim.sock unix:///run/containerd/
containerd.sock unix:///run/crio/crio.sock unix:///var/run/cri-dockerd.sock
```

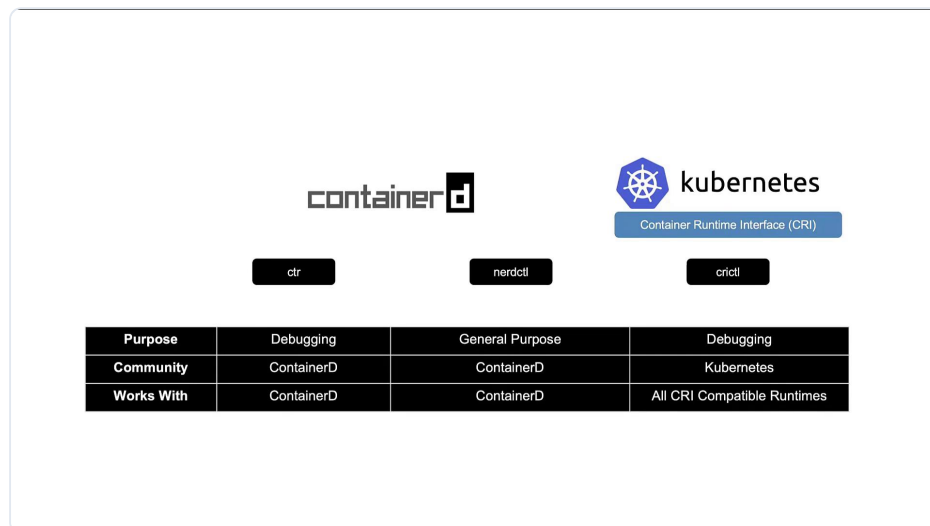
With Kubernetes 1.24, significant changes were introduced. The docker-socket was replaced

```
```bash theme={null}
$ crictl --runtime-endpoint <endpoint>
$ export CONTAINER_RUNTIME_ENDPOINT=<endpoint>
```

For further details, refer to the [Kubernetes CLI Tools repository](#) (see pull request 869 and issue 868).

Summary

- The CTR tool, bundled with ContainerD, is primarily used for debugging.
- NerdCTL offers a Docker-like CLI for general-purpose container operations and advanced ContainerD features.
- Crictl, maintained by the Kubernetes community, is designed for interacting with all CRI-compatible container runtimes and is mainly used for debugging and inspection.



Each tool serves a distinct role within the container ecosystem. While CTR and crictl focus on debugging, NerdCTL is the recommended tool for routine container management tasks, offering a familiar interface for users transitioning from Docker CLI commands.

Thank you for following along in this lesson on Docker versus ContainerD and the accompanying CLI tools. Enjoy exploring container runtimes and enhancing your container management workflows!

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

A note on Docker Deprecation

The article clarifies Docker's deprecation in Kubernetes while highlighting its continued relevance in development and container management.

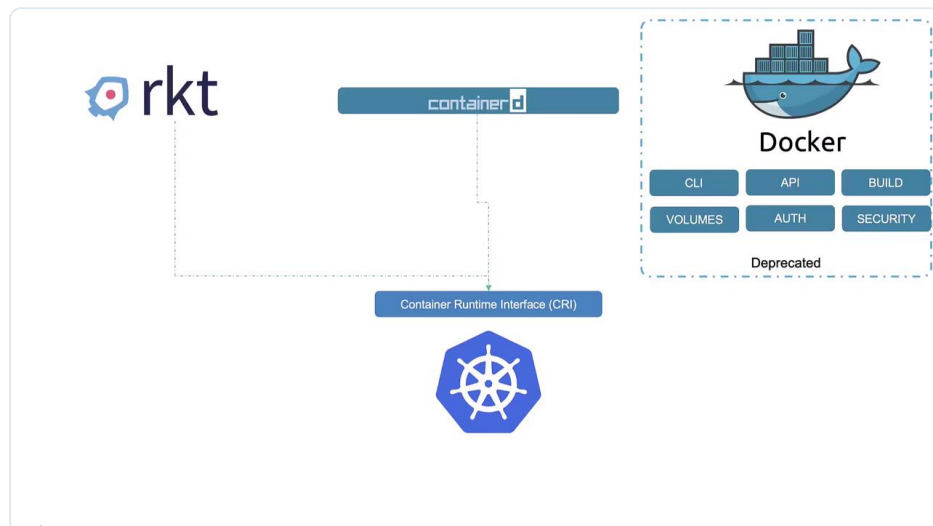
In this lesson, we address common questions regarding Docker's deprecation and clarify why Docker remains relevant despite its deprecated status in Kubernetes. This explanation aims to resolve the confusion many students experience.

Background

Originally, Docker was the sole supported container runtime for Kubernetes. To enable compatibility with other container runtimes, the Container Runtime Interface (CRI) was introduced. Docker, as a complete platform, bundled several tools together, including:

- The Docker CLI
- The API
- Build tools for creating images
- Volume support
- Security configurations
- The container runtime (runc)
- The daemon (containerd) that managed the runtime

Containerd is now CRI-compatible and functions as a standalone component that interacts directly with Kubernetes. Thanks to this change, Kubernetes no longer depends on Docker's additional tools since it manages container operations natively.



Docker's Role in Modern Development

While Kubernetes has officially deprecated Docker as its runtime, Docker remains the most popular container solution for day-to-day development and build processes. Kubernetes simply leverages containerd for orchestration but many development practices still benefit from Docker's extensive toolset.

Even though Kubernetes doesn't require Docker for container orchestration, using Docker to understand container fundamentals remains beneficial. Once these basics are mastered, shifting focus to Kubernetes and containerd becomes more intuitive.

Throughout the course, Docker is used as a primary example to explain container concepts. This strategy allows learners to grasp fundamental ideas before transitioning to Kubernetes-specific orchestration. If Docker is not installed on your system or if you prefer using containerd, you can convert Docker commands to their equivalent [kubectl](#) commands as needed.

This clarification sets the context for the remainder of the course and ensures a smooth transition from understanding container concepts with Docker to applying them within Kubernetes environments.

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

ETCD for Beginners

This article introduces etcd, a distributed key-value store, covering its operation, installation, and transition from API v2 to v3.

This article provides a concise introduction to etcd—a distributed, reliable key-value store that is both simple and fast. Whether you're new to key-value storage concepts or already familiar with etcd's role in Kubernetes environments, this guide offers valuable insights into its operation and quick start instructions using the etcdctl client tool.

Objectives

- What is ETCD?
- What is a Key-Value Store?
- How to get started quickly?
- How to operate ETCD?

Later:

- What is a distributed system?
- How ETCD Operates
- RAFT Protocol
- Best practices on number of nodes

Later in the article, we will dive deeper into high availability topics. You'll learn about distributed systems, how etcd operates in cluster mode, the Raft consensus protocol, and best practices for configuring a resilient etcd cluster.

What Is a Key-Value Store?

Traditional relational databases, such as SQL databases, store data in tables with rows and columns. For example, a table that contains information about individuals might look like this:

- Each row represents a single person.

- Each column holds a specific detail about that person (e.g., name, age).

If you need to include additional information (like salary data for employed individuals or grades for students), you must expand the table. This means adding columns that may not apply universally to every row.

In contrast, a key-value store organizes data as independent documents or files. Each document contains all relevant information for an individual, allowing flexible and dynamic data structures:

- Working individuals can have documents with salary details.
- Students can have documents with grade details.

key-value store

Key	Value
Name	John Doe
Age	45
Location	New York
Salary	5000

Key	Value
Name	Dave Smith
Age	34
Location	New York
Salary	4000

Key	Value
Name	Aryan Kumar
Age	10
Location	New York
Grade	A

Key	Value
Name	Lauren Rob
Age	13
Location	Bangalore
Grade	C

Key	Value
Name	Lily Oliver
Age	15
Location	Bangalore
Grade	B

For complex data transactions, structured formats like JSON or YAML are used. Here are some examples of key-value pairs stored as JSON documents:

```
```json theme={null} { "name": "John Doe", "age": 45, "location": "New York", "salary": 5000 }
```

```
```json theme={null}
{
  "name": "Dave Smith",
  "age": 34,
  "location": "New York",
  "salary": 4000,
  "organization": "ACME"
}
```

```
```json theme={null} { "name": "Aryan Kumar", "age": 10, "location": "New York", "Grade": "A" }
```

```
```json theme={null}
{
  "name": "Lily Oliver",
  "age": 15,
  "location": "Bangalore",
  "Grade": "B"
}
```

```
```json theme={null} { "name": "Lauren Rob", "age": 13, "location": "Bangalore",
"Grade": "C" }
```

### ## Installing and Getting Started with etcd

Getting started with etcd is straightforward. First, download the correct binary for your system.

For example, you can download etcd using the following command:

```
```bash theme={null}
curl -L https://github.com/etcd-io/etcd/releases/download/v3.3.11/etcd-v3.3.11-linux-amd64.
```

After extracting and starting etcd, the accompanying command-line client, etcdctl, lets you interact with your etcd instance. To store a key-value pair using the etcd API v2, run:

```
```bash theme={null} ./etcdctl set key1 value1
```

And to retrieve it:

```
```bash theme={null}
./etcdctl get key1
```

If you run the etcdctl command without any arguments, it displays a list of available commands similar to this:

```
```bash theme={null} ./etcdctl
```

```
```text theme={null}
NAME:
etcdctl - A simple command line client for etcd.

COMMANDS:
  backup      backup an etcd directory
  cluster-health check the health of the etcd cluster
  mk          make a new key with a given value
  mkdir       make a new directory
  rm          remove a key or a directory
  rmdir       removes the key if it is an empty directory or a key-value pair
  get         retrieve the value of a key
```

Different articles may reference varying versions of etcd. Understanding its version history—starting with version 0.1 in August 2013, introducing the Raft

consensus algorithm in version 2.0 (February 2015), and further optimizations in version 3 (January 2017)—can clarify command differences. The etcd project was incubated in the CNCF in November 2018.



Transitioning from API v2 to v3

One critical change between etcd versions is the API version used by etcdctl. Although etcdctl may be configured to use API v2 by default, newer installations typically default to API v3. You can verify the API version by running:

```
```bash theme={null} ./etcdctl --version
```

This command might output:

```
```text theme={null}
etcdctl version: 3.3.11
API version: 2
```

When you list available commands in API v2 mode:

```
```bash theme={null} ./etcdctl
```

the command list includes options like:

```
```text theme={null}
COMMANDS:
  backup          backup an etcd directory
  cluster-health  check the health of the etcd cluster
  mk              make a new key with a given value
  mkdir           make a new directory
  rm              remove a key or a directory
  rmdir           removes the key if it is an empty directory or a key-value pair
  get             retrieve the value of a key
  ls             retrieve a directory
  set             set the value of a key
  setdir          create a new directory or update an existing directory TTL
  update          update an existing key with a given value
  updatedir       update an existing directory
  watch           watch a key for changes
  exec-watch      watch a key for changes and exec an executable
  member          member add, remove and list subcommands
  user            user add, grant and revoke subcommands
  role            role add, grant and revoke subcommands
```

To switch etcdctl to use API v3, you have two options:

1. Prepend the environment variable to the command:

```
bash theme={null} ETCDCCTL_API=3 ./etcdctl version
```

This produces the output:

```
text theme={null} etcdctl version: 3.3.11 API version: 3
```

1. Or, export the environment variable for your session:

```
bash theme={null} export ETCDCCTL_API=3 ./etcdctl version
```

The output will similarly confirm the switch to API v3.

In API v3, the command for setting a key changes from "set" to "put", while retrieving a key remains the same ("get"). Note that the `version` command is now a subcommand rather than an option.

After setting the environment variable, you can perform key-value operations with API v3 commands. For example:

```
```bash theme={null} export ETCDCCTL_API=3 ./etcdctl version
```

Expected output:

```
```text theme={null}
etcdctl version: 3.3.11
API version: 3
```

Then, set a key-value pair:

```
```bash theme={null} ./etcdctl put key1 value1
```

This should output:

```
```text theme={null}  
OK
```

And retrieve the value:

```
```bash theme={null} ./etcdctl get key1
```

Expected output:

```
```text theme={null}  
key1  
value1
```

Conclusion

In this article, we explored etcd and the fundamentals of key-value storage, contrasting it with traditional relational databases. We demonstrated how to install etcd, use the etcdctl command-line tool, and navigate the transition from API v2 to API v3. In our upcoming article, we will examine how to configure etcd in a high-availability environment and its seamless integration with Kubernetes.

Thank you for reading, and stay tuned for further insights into etcd and distributed systems.

Documentation Index

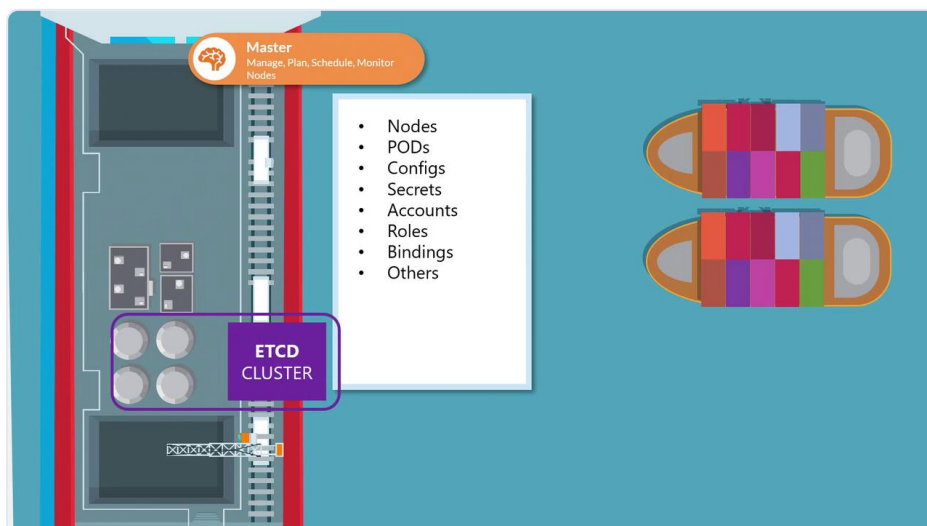
Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

ETCD in Kubernetes

This article explores the role of etcd in Kubernetes, covering deployment methods and high availability considerations.

Welcome to this comprehensive guide on etcd in Kubernetes. In this article, we explore the critical role of etcd in storing cluster state, detail different deployment approaches, and explain high availability considerations. Whether you're setting up a Kubernetes cluster from scratch or using kubeadm, understanding etcd is essential.

etcd is a distributed key-value store that maintains configuration data, state information, and metadata for your Kubernetes cluster. Every object—nodes, pods, configurations, secrets, accounts, roles, and role bindings—is stored within etcd. When you run a command like `kubectl get`, the data is retrieved from this data store.



Any changes you make to the cluster—whether adding nodes, deploying pods, or configuring ReplicaSets—are first recorded in etcd. Only after etcd is updated are these changes considered to be complete.

The etcd server typically listens on port 2379 for client requests. Ensuring that the advertised client URL (via the `--advertise-client-urls` option) is

correctly configured is crucial for proper communication between the Kubernetes API Server and etcd.

Deployment Methods

Depending on your Kubernetes setup, you can deploy etcd in two primary ways: manually from scratch or automatically with kubeadm. Each method has its use cases, with manual setups providing a deeper understanding of etcd configurations and kubeadm streamlining the deployment process.

Deploying etcd from Scratch

When setting up your cluster manually, you'll need to download the etcd binaries, install them, and configure etcd as a service on your master node. Manual deployment gives you more control over configuration options, particularly for setting up TLS certificates.

Below is an example of how you might download the etcd binaries and configure the etcd service:

```
```bash theme={null} wget -q --https-only \ "https://github.com/coreos/etcd/releases/download/v3.3.9/etcd-v3.3.9-linux-amd64.tar.gz"
```

## Example etcd service configuration

---

```
ExecStart=/usr/local/bin/etcd \ --name ${ETCD_NAME} \ --cert-file=/etc/etcd/
kubernetes.pem \ --key-file=/etc/etcd/kubernetes-key.pem \ --peer-cert-file=/
etc/etcd/kubernetes.pem \ --peer-key-file=/etc/etcd/kubernetes-key.pem \ --
trusted-ca-file=/etc/etcd/ca.pem \ --peer-trusted-ca-file=/etc/etcd/ca.pem \ --
peer-client-cert-auth \ --client-cert-auth \ --initial-advertise-peer-urls https://$
${INTERNAL_IP}:2380 \ --listen-peer-urls https://${INTERNAL_IP}:2380 \ --
listen-client-urls https://${INTERNAL_IP}:2379,https://127.0.0.1:2379 \ --
advertise-client-urls https://${INTERNAL_IP}:2379 \ --initial-cluster-token etcd-
cluster-0 \ --initial-cluster controller-0=https://$
${CONTROLLER0_IP}:2380,controller-1=https://${CONTROLLER1_IP}:2380 \
--initial-cluster-state new \ --data-dir=/var/lib/etcd
```

```
<Callout icon="lightbulb" color="#1CB2FE">
```

```
 For detailed information on configuring TLS certificates, refer to the Kubernetes document
</Callout>
```

```

```

## ## High Availability Considerations

In a production Kubernetes environment, high availability (HA) is paramount. By running mul

To enable HA, each etcd instance must know about its peers. This is achieved by configuring

```
```bash theme={null}
```

```
ExecStart=/usr/local/bin/etcd \
```

```
--name ${ETCD_NAME} \
```

```
--cert-file=/etc/etcd/kubernetes.pem \
```

```
--key-file=/etc/etcd/kubernetes-key.pem \
```

```
--peer-cert-file=/etc/etcd/kubernetes.pem \
```

```
--peer-key-file=/etc/etcd/kubernetes-key.pem \
```

```
--trusted-ca-file=/etc/etcd/ca.pem \
```

```
--peer-trusted-ca-file=/etc/etcd/ca.pem \
```

```
--peer-client-cert-auth \
```

```
--client-cert-auth \
```

```
--initial-advertise-peer-urls=https://${INTERNAL_IP}:2380 \
```

```
--listen-peer-urls=https://${INTERNAL_IP}:2380 \
```

```
--advertise-client-urls=https://${INTERNAL_IP}:2379 \
```

```
--initial-cluster-token=etcd-cluster-0 \
```

```
--initial-cluster controller-0=https://${CONTROLLER0_IP}:2380,controller-1=https://${CO
```

```
--initial-cluster-state=new \
```

```
--data-dir=/var/lib/etcd
```

In some deployments, you may use separate certificate files for peer communications (e.g., `/etc/etcd/peer.pem` and `/etc/etcd/peer-key.pem`). Always tailor these settings to match your desired security posture.

Deploying etcd with kubeadm

For many test environments and streamlined deployments, kubeadm automatically configures etcd. When you use kubeadm, the etcd server runs as a pod within the kube-system namespace, abstracting away the manual setup details.

To view all the pods running in the kube-system namespace, including etcd, run:

```
```bash theme={null} kubectl get pods -n kube-system
```

Example output:

```
```text theme={null}
NAMESPACE          NAME                                READY   STATUS    RESTARTS   AGE
kube-system        coredns-78fcd6894-prwl             1/1     Running   0           1h
kube-system        coredns-78fcd6894-vqd9w            1/1     Running   0           1h
kube-system        etcd-master                         1/1     Running   0           1h
kube-system        kube-apiserver-master               1/1     Running   0           1h
kube-system        kube-controller-manager-master      1/1     Running   0           1h
kube-system        kube-proxy-f6k26                   1/1     Running   0           1h
kube-system        kube-proxy-hnzw                     1/1     Running   0           1h
kube-system        kube-scheduler-master               1/1     Running   0           1h
kube-system        weave-net-924k8                     2/2     Running   1           1h
kube-system        weave-net-hzfcz                     2/2     Running   1           1h
```

To examine the keys stored in etcd (organized under the registry directory), use the following command:

```
```bash theme={null} kubectl exec etcd-master -n kube-system -- etcdctl get / --
prefix --keys-only
```

Sample output:

```
```text theme={null}
/registry/apiregistration.k8s.io/apiservices/v1
/registry/apiregistration.k8s.io/apiservices/v1.apps
/registry/apiregistration.k8s.io/apiservices/v1.authentication.k8s.io
/registry/apiregistration.k8s.io/apiservices/v1.authorization.k8s.io
/registry/apiregistration.k8s.io/apiservices/v1.autoscaling
/registry/apiregistration.k8s.io/apiservices/v1.batch
/registry/apiregistration.k8s.io/apiservices/v1.networking.k8s.io
/registry/apiregistration.k8s.io/apiservices/v1.rbac.authorization.k8s.io
/registry/apiregistration.k8s.io/apiservices/v1beta1.admissionregistration.k8s.io
```

The etcd root directory, organized as the registry, contains subdirectories for various Kubernetes components such as nodes, pods, ReplicaSets, and Deployments.

Conclusion

In this article, we examined etcd's essential role in Kubernetes by exploring both manual deployment and automated configuration with kubeadm. We also discussed best practices for configuring high availability in multi-master environments. As you advance, further exploration into HA configurations and enhanced security measures will help you operate a robust and resilient Kubernetes cluster.

For more in-depth resources, check out:

- [Kubernetes Documentation](#)

- [Kubernetes Basics](#)
- [Docker Hub](#)
- [Terraform Registry](#)

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Kube API Server

This article provides a comprehensive guide on the Kube API Servers role in managing requests and coordinating components in a Kubernetes cluster.

Welcome to this comprehensive guide on the Kube API Server in Kubernetes. In this article, we explore how the Kube API Server acts as the central management component in a Kubernetes cluster by handling requests from kubectl, validating and authenticating them, interfacing with the etcd datastore, and coordinating with other system components.

When you execute a command like:

```
``bash theme={null} kubectl get nodes
```

the utility sends a request to the API Server. The server processes this request by authenticating the request.

```
``plaintext theme={null}
```

NAME	STATUS	ROLES	AGE	VERSION
master	Ready	master	20m	v1.11.3
node01	Ready	<none>	20m	v1.11.3

API Server Request Lifecycle

When a direct API POST request is made to create a pod, the API Server:

1. Authenticates and validates the request.
2. Constructs a pod object (initially without a node assignment) and updates the etcd store.
3. Notifies the requester that the pod has been created.

For instance, using a curl command:

```
``bash theme={null} curl -X POST /api/v1/namespaces/default/pods ...[other]
Pod created!
```

The scheduler continuously monitors the API Server for pods that need node assignments. Once

<Callout icon="lightbulb" color="#1CB2FE">

At the heart of these operations is the Kube API Server, ensuring secure and validated co

</Callout>

<Frame>

! [The image lists six steps related to the Kube-api Server: Authenticate User, Validate

</Frame>

Deployment and Setup

If your cluster is bootstrapped with a kube admin tool, most of these intricate details are

Typical Service Configuration

The Kube API Server is launched with a variety of parameters to secure communication and ma

```
```bash theme={null}
```

```
wget https://storage.googleapis.com/kubernetes-release/release/v1.18.0/bin/linux/amd64/kube
```

```
kube-apiserver.service
```

```
ExecStart=/usr/local/bin/kube-apiserver \\\
```

```
--advertise-address=${INTERNAL_IP} \\\
```

```
--allow-privileged=true \\\
```

```
--apiserver-count=3 \\\
```

```
--authorization-mode=Node,RBAC \\\
```

```
--bind-address=0.0.0.0 \\\
```

```
--client-ca-file=/var/lib/kubernetes/ca.pem \\\
```

```
--enable-admission-plugins=Initializers,NamespaceLifecycle,NodeRestriction,LimitRanger,Se
```

```
--enable-swagger-ui=true \\\
```

```
--etcd-cafile=/var/lib/kubernetes/ca.pem \\\
```

```
--etcd-certfile=/var/lib/kubernetes/kubernetes.pem \\\
```

```
--etcd-keyfile=/var/lib/kubernetes/kubernetes-key.pem \\\
```

```
--etcd-servers=https://127.0.0.1:2379 \\\
```

```
--event-ttl=1h \\\
```

```
--experimental-encryption-provider-config=/var/lib/kubernetes/encryption-config.yaml \\\
```

```
--kubelet-certificate-authority=/var/lib/kubernetes/ca.pem \\\
```

```
--kubelet-client-certificate=/var/lib/kubernetes/kubernetes.pem \\\
```

```
--kubelet-client-key=/var/lib/kubernetes/kubernetes-key.pem \\\
```

```
--kubelet-https=true \\\
```

```
--runtime-config=api/all \\\
```

```
--service-account-key-file=/var/lib/kubernetes/service-account.pem \\\
```

```
--service-cluster-ip-range=10.32.0.0/24 \\\
```

```
--service-node-port-range=30000-32767 \\\
```

```
--v=2
```

The configuration includes several certificate-related options, securing communication channels between various Kubernetes components. In upcoming sections, we will take a deeper look at SSL/TLS certificates and their role in ensuring secure interactions.

## Verifying the Deployment

For clusters set up with kube-admin tools, the Kube API Server is deployed as a pod in the kube-system namespace. To inspect these pods, run:

```
```bash theme={null} kubectl get pods -n kube-system
```

Expected output may include:

```
```plaintext theme={null}
```

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE
kube-system	coredns-78fcd6894-hwrq9	1/1	Running	0	16m
kube-system	coredns-78fcd6894-rzhjr	1/1	Running	0	16m
kube-system	etcd-master	1/1	Running	0	15m
kube-system	kube-apiserver-master	1/1	Running	0	15m
kube-system	kube-controller-manager-master	1/1	Running	0	15m
kube-system	kube-proxy-lzt6f	1/1	Running	0	16m
kube-system	kube-proxy-zm5qd	1/1	Running	0	15m
kube-system	kube-scheduler-master	1/1	Running	0	15m
kube-system	weave-net-29z42	2/2	Running	1	16m
kube-system	weave-net-snm1l	2/2	Running	1	16m

For non-kube-admin setups, you can examine the container command options directly within the pod manifest. Here's an excerpt from a pod definition:

```
```yaml theme={null} spec: containers: - command: - kube-apiserver - --
authorization-mode=Node,RBAC - --advertise-address=172.17.0.32 - --allow-
privileged=true - --client-ca-file=/etc/kubernetes/pki/ca.crt - --disable-
admission-plugins=PersistentVolumeLabel - --enable-admission-
plugins=NodeRestriction - --enable-bootstrap-token-auth=true - --etcd-cafile=/
etc/kubernetes/pki/etcd/ca.crt - --etcd-certfile=/etc/kubernetes/pki/apiserver-
etcd-client.crt - --etcd-keyfile=/etc/kubernetes/pki/apiserver-etcd-client.key - --
etcd-servers=https://127.0.0.1:2379 - --insecure-port=0 - --kubelet-client-
certificate=/etc/kubernetes/pki/apiserver-kubelet-client.crt - --kubelet-client-
key=/etc/kubernetes/pki/apiserver-kubelet-client.key - --kubelet-preferred-
address-types=InternalIP,ExternalIP,Hostname - --proxy-client-cert-file=/etc/
kubernetes/pki/front-proxy-client.crt - --proxy-client-key-file=/etc/kubernetes/
pki/front-proxy-client.key - --requestheader-allowed-names=front-proxy-client -
--requestheader-client-ca-file=/etc/kubernetes/pki/front-proxy-ca.crt - --
requestheader-extra-headers-prefix=X-Remote-Extra- - --requestheader-
group-headers=X-Remote-Group - --requestheader-username-headers=X-
Remote-User
```

Another way to review the active API Server configuration is by checking the systemd service

```
```bash theme={null}
cat /etc/systemd/system/kube-apiserver.service
```

An example excerpt from this file might be:

```

bash theme={null} [Service] ExecStart=/usr/local/bin/kube-apiserver
\\ --advertise-address=${INTERNAL_IP} \\ --allow-privileged=true \\
--apiserver-count=3 \\ --audit-log-maxage=30 \\ --audit-log-
maxbackup=3 \\ --audit-log-maxsize=100 \\ --audit-log-path=/var/log/
audit.log \\ --authorization-mode=Node,RBAC \\ --bind-
address=0.0.0.0 \\ --client-ca-file=/var/lib/kubernetes/ca.pem \\ --
enable-admission-
plugins=Initializers,NamespaceLifecycle,NodeRestriction,LimitRanger,ServiceAccount,DefaultStorageClass
\\ --enable-swagger-ui=true \\ --etcd-cafile=/var/lib/kubernetes/
ca.pem \\ --etcd-certfile=/var/lib/kubernetes/kubernetes.pem \\ --
etcd-keyfile=/var/lib/kubernetes/kubernetes-key.pem \\ --etcd-
servers=https://10.240.0.10:2379,https://10.240.0.11:2379,https://
10.240.0.12:2379 \\ --event-ttl=1h \\ --experimental-encryption-
provider-config=/var/lib/kubernetes/encryption-config.yaml \\ --
kubelet-certificate-authority=/var/lib/kubernetes/ca.pem \\ --
kubelet-client-certificate=/var/lib/kubernetes/kubernetes.pem \\ --
kubelet-client-key=/var/lib/kubernetes/kubernetes-key.pem \\ ...

```

## Quick Reference Table

Below is a summary of some key Kubernetes components involved in the Kube API Server workflow:

Component	Role	Command/Action Example
kubectl	CLI tool to send API requests	<code>kubectl get nodes</code>
Kube API Server	Central component for processing, authenticating, and validating requests	Processes API requests and interacts with etcd
Scheduler	Monitors API Server for unassigned pods and assigns them to worker nodes	Automatically assigns node to newly created pods
Kubelet	Runs on worker nodes to manage pod lifecycle and communicate status back to API Server	Interacts with container runtime to deploy images
etcd	Distributed key-value store used for saving cluster configuration	Stores all cluster state data

## Summary

In this article, we provided an overview of the Kube API Server, its interactions with other essential components, and various methods to inspect its configuration—both through pod manifests and systemd service files. In

subsequent sections, we will delve deeper into certificate management, including SSL/TLS configurations, to reinforce secure communications within your Kubernetes cluster.

For a deeper understanding of Kubernetes and its components, explore the [Kubernetes Documentation](#).

This concludes our discussion on the Kube API Server.

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Kube Controller Manager

This guide covers the Kube Controller Managers role, configuration, and importance in managing controllers within a Kubernetes cluster.

Welcome to this comprehensive guide on the Kube Controller Manager, a vital component in Kubernetes responsible for managing a variety of controllers within your cluster. Understanding its role and configuration is crucial for ensuring a resilient and well-orchestrated Kubernetes environment.

## Overview

In Kubernetes, a controller acts like a department in an organization—each controller is tasked with handling a specific responsibility. For instance, one controller might monitor the health of nodes, while another ensures that the desired number of pods is always running. These controllers constantly observe system changes to drive the cluster toward its intended state.

The Node Controller, for example, checks node statuses every five seconds through the Kube API Server. If a node stops sending heartbeats, it is not immediately marked as unreachable; instead, there is a grace period of 40 seconds followed by an additional five minutes for potential recovery before its pods are rescheduled onto a healthy node.

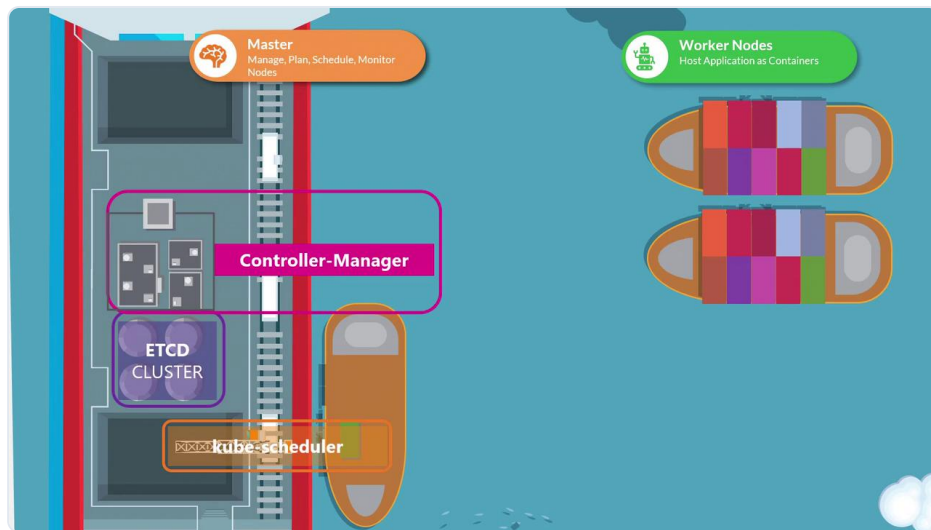
### Example: Checking Node Statuses

```
```bash theme={null} kubectl get nodes NAME STATUS ROLES AGE VERSION worker-1 Ready 8d v1.13.0 worker-2 Ready 8d v1.13.0
```

In the case where a node fails to recover, the output might look like this:

```
```bash theme={null}
kubectl get nodes
NAME STATUS ROLES AGE VERSION
worker-1 Ready <none> 8d v1.13.0
worker-2 NotReady <none> 8d v1.13.0
```

Another essential controller is the Replication Controller, which ensures that the specified number of pods is maintained by creating new pods when needed. This mechanism reinforces the resilience and reliability of your Kubernetes cluster.



All core Kubernetes constructs—such as Deployments, Services, Namespaces, and Persistent Volumes—rely on these controllers. Essentially, controllers serve as the "brains" behind many operations in a Kubernetes cluster.

## How Controllers Are Packaged

All individual controllers are bundled into a single process known as the Kubernetes Controller Manager. When you deploy the Controller Manager, every associated controller is started together. This unified deployment simplifies management and configuration.

## Installing and Configuring the Kube Controller Manager

To install and view the Kube Controller Manager, follow these steps:

1. Download the Kube Controller Manager from the Kubernetes release page.
2. Extract the binary and run it as a service.
3. Review the configurable options provided, which allow you to tailor its behavior.

### Downloading the Controller Manager

```
```bash theme={null} wget https://storage.googleapis.com/kubernetes-release/release/v1.13.0/bin/linux/amd64/kube-controller-manager
```

Sample Service Configuration

Below is an example of a service file (`kube-controller-manager.service`) used to run the C

```

```bash theme={null}
ExecStart=/usr/local/bin/kube-controller-manager \
 --address=0.0.0.0 \
 --cluster-cidr=10.200.0.0/16 \
 --cluster-name=kubernetes \
 --cluster-signing-cert-file=/var/lib/kubernetes/ca.pem \
 --cluster-signing-key-file=/var/lib/kubernetes/ca-key.pem \
 --kubeconfig=/var/lib/kubernetes/kube-controller-manager.kubeconfig \
 --leader-elect=true \
 --root-ca-file=/var/lib/kubernetes/ca.pem \
 --service-account-private-key-file=/var/lib/kubernetes/service-account-key.pem \
 --service-cluster-ip-range=10.32.0.0/24 \
 --use-service-account-credentials=true \
 --v=2

```

This configuration includes additional options for the Node Controller, such as node monitor period, grace period, and eviction timeout. Additionally, you can control which controllers are enabled through the `--controllers` flag.

By default, all controllers are enabled. You can selectively enable or disable controllers by using the syntax `foo` to enable and `-foo` to disable. For example, `--controllers=*,-tokencleaner` will disable the `tokencleaner` controller.

### Example of Specifying Controllers

```

```bash theme={null} --controllers stringSlice Default: [] A list of controllers to
enable. " enables all on-by-default controllers, 'foo' enables the controller
named 'foo', '-foo' disables the controller named 'foo'. All controllers:
attachdetach, bootstrapsigner, clusterrole-aggregation, cronjob, csrapproving,
csrcleaner, csrsigning, daemonset, deployment, disruption, endpoint,
garbagecollector, horizontalpodautoscaling, job, namespace, nodeipam,
nodelifecycle, persistentvolume-binder, persistentvolume-expander, podgc, pv-
protection, pvc-protection, replicaset, replicationcontroller, resourcequota, root-
ca-cert-publisher, route, service, serviceaccount, serviceaccount-token,
statefulset, tokencleaner, ttl, ttl-after-finished Disabled-by-default controllers:
bootstrapsigner, tokencleaner

```

Viewing the Controller Manager in Action

Depending on your cluster setup, the Controller Manager may run as a pod in the `kube-system` namespace.

Service Configuration Example (Non-Kubeadm Environments)

```

```plaintext theme={null}
[Service]
ExecStart=/usr/local/bin/kube-controller-manager \
 --address=0.0.0.0 \
 --cluster-cidr=10.200.0.0/16 \
 --cluster-name=kubernetes \
 --cluster-signing-cert-file=/var/lib/kubernetes/ca.pem \
 --cluster-signing-key-file=/var/lib/kubernetes/ca-key.pem \
 --kubeconfig=/var/lib/kubernetes/kube-controller-manager.kubeconfig \
 --leader-elect=true \
 --root-ca-file=/var/lib/kubernetes/ca.pem \
 --service-account-private-key-file=/var/lib/kubernetes/service-account-key.pem \
 --service-cluster-ip-range=10.32.0.0/24 \
 --use-service-account-credentials=true \
 --v=2
Restart=on-failure
RestartSec=5

```

## Checking the Running Process

To verify that the Kube Controller Manager is running and to inspect its active options, execute the following command on the master node:

```
```plaintext theme={null} ps -aux | grep kube-controller-manager
```

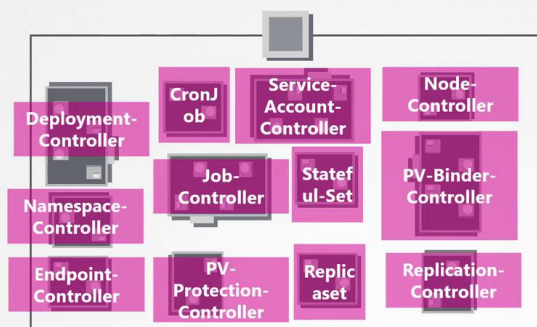
An example output might be:

```

```plaintext theme={null}
root 1994 2.7 5.1 154360 105024 ? Ssl 06:45 1:25 kube-controller-manager

```

## Controller



- Watch Status
- Remediate Situation
- Node Monitor Period = 5s
- Node Monitor Grace Period = 40s
- POD Eviction Timeout = 5m

## Conclusion

---

This guide has provided an in-depth look at the Kube Controller Manager, detailing its critical functions in managing controllers, monitoring system changes, and ensuring the desired state within your Kubernetes cluster. By understanding and properly configuring the Controller Manager, you play a key role in maintaining a robust and scalable environment.

Stay tuned for more lessons to further enhance your Kubernetes expertise!

For additional details, you might find these resources useful:

- [Kubernetes Documentation](#)
- [Kubernetes Basics](#)

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Kube Scheduler

This article provides a comprehensive overview of the Kube Scheduler's role in pod placement within Kubernetes.

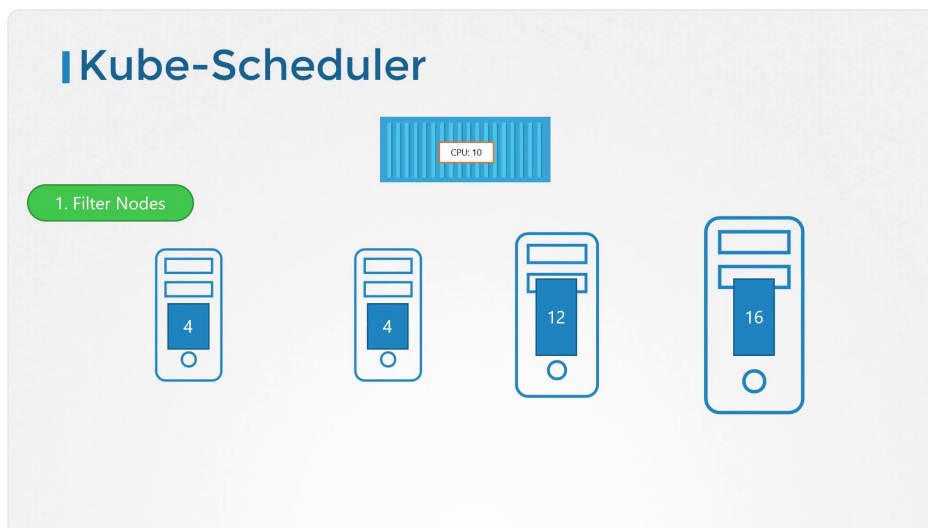
Welcome to this comprehensive lesson on the Kube Scheduler, a core component of Kubernetes. In this guide, we delve into the scheduler's role in determining on which node a pod should be placed. It is important to note that while the scheduler makes the placement decision, the actual creation of the pod on the selected node is carried out by the Kubelet.

## Scheduler Process Overview

The primary responsibility of the Kubernetes scheduler is to assign pods to nodes based on a series of criteria. This ensures that the selected node has sufficient resources and meets any specific requirements. For instance, different nodes may be designated for certain applications or come with varied resource capacities. When multiple pods and nodes are involved, the scheduler assesses each pod against the available nodes through a two-phase process: filtering and ranking.

### 1. Filtering Phase

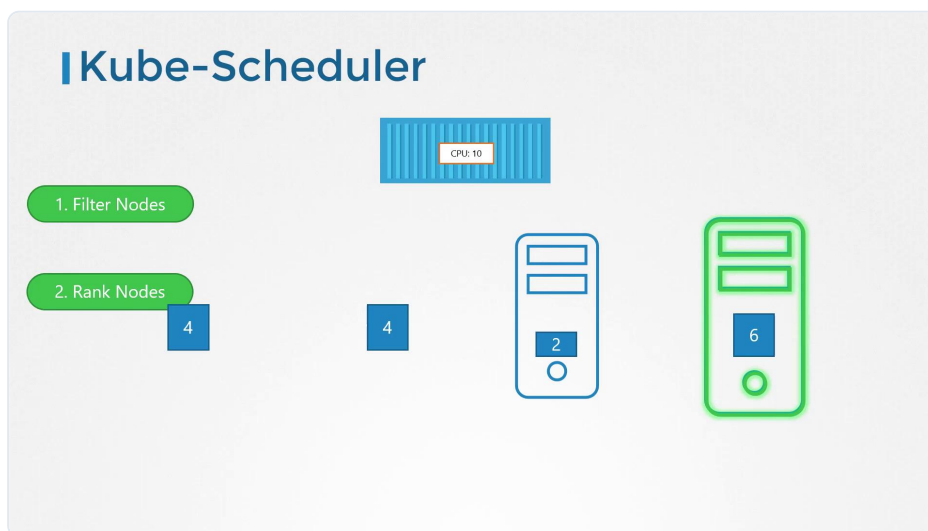
In the filtering phase, the scheduler eliminates nodes that do not meet the pod's resource requirements. For example, nodes that lack sufficient CPU or memory are immediately excluded.



As depicted above, the diagram demonstrates the elimination of nodes with insufficient resources, leaving only the candidate nodes that can accommodate the pod's needs.

## 2. Ranking Phase

After filtering, the scheduler enters the ranking phase. Here, it uses a priority function to score and compare the remaining nodes on a scale from 0 to 10, ultimately selecting the best match. For instance, if placing a pod on one node would leave six free CPUs (four more than an alternative node), that node is assigned a higher score and is chosen.



This high-level overview outlines how Kubernetes efficiently filters and ranks nodes for optimal pod placement. The scheduler's design is highly customizable, allowing you to develop your own scheduler if the need arises. For more advanced scheduling configurations—such as resource limits, taints and tolerations, node selectors, and affinity rules—refer to the [Kubernetes Documentation](#).

Customizing the scheduling process can help tailor your Kubernetes environment to meet specific workloads and performance requirements.

## Installing and Running the Kube Scheduler

To install the kube-scheduler, download the binary from the Kubernetes release page. Once downloaded and extracted, you can run it as a service by specifying the scheduler configuration file. Below is a sample command for downloading the binary and an example systemd service configuration:

```
```bash theme={null} wget https://storage.googleapis.com/kubernetes-release/release/v1.13.0/bin/linux/amd64/kube-scheduler
```

Below is an example of the systemd service configuration for the kube-scheduler:

```
```bash theme={null}
File: kube-scheduler.service
ExecStart=/usr/local/bin/kube-scheduler \
 --config=/etc/kubernetes/config/kube-scheduler.yaml \
 --v=2
```

If you are using the kubeadm tool to set up your cluster, kubeadm deploys the kube-scheduler as a pod in the `kube-system` namespace on the master node. You can inspect the scheduler configuration by viewing the pod manifest file:

```
```bash theme={null} cat /etc/kubernetes/manifests/kube-scheduler.yaml
```

This manifest file outlines the options used during the scheduler's deployment. To verify t

```
```bash theme={null}
ps -aux | grep kube-scheduler
```

An example output might look similar to:

```
bash theme={null} root 2477 0.8 1.6 48524 34044 ? Ssl 17:31 0:08
kube-scheduler --address=127.0.0.1 --kubeconfig=/etc/kubernetes/
scheduler.conf --leader-elect=true
```

If you need more detailed configuration options or troubleshooting tips for the kube-scheduler, refer to the [Kubernetes Documentation](#).

This concludes our in-depth lesson on the Kube Scheduler. In future modules, we will explore advanced scheduling concepts and configurations to further enhance your Kubernetes deployment strategies.

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

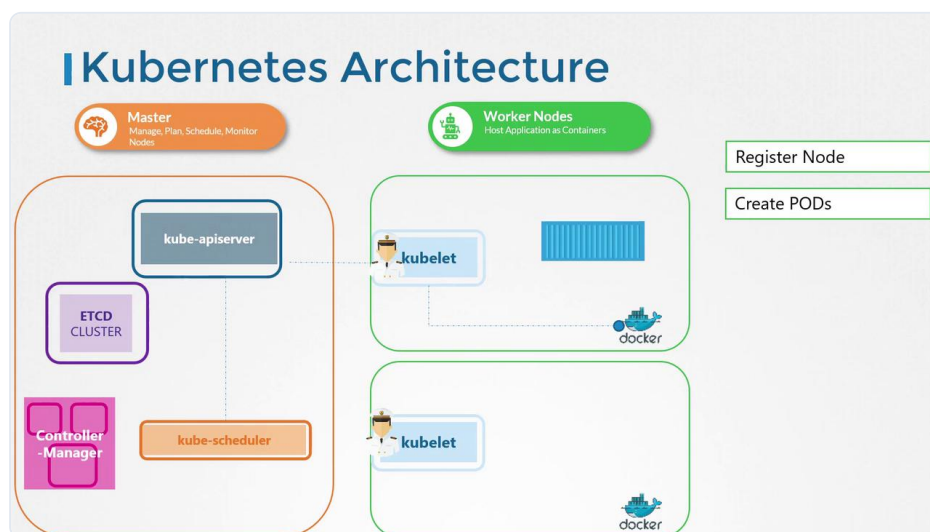
# Kubelet

This article covers the Kubelets responsibilities, its role in Kubernetes, and instructions for installing it on worker nodes.

Welcome to this lesson on the Kubelet. In this guide, you will learn about the responsibilities of the Kubelet, its role within the Kubernetes cluster, and how to install it on worker nodes.

The Kubelet is often described as the "captain of the ship." It oversees node activities by managing container operations such as starting and stopping containers based on instructions from the master scheduler. Additionally, the Kubelet registers the node with the Kubernetes cluster and continuously monitors the state of pods and their containers. It regularly reports the status of the node and its workloads to the Kubernetes API server.

When the Kubelet receives instructions to run a container or pod, it communicates with the container runtime (e.g., Docker) to download the required image and initiate the container. It then maintains the health of these containers and ensures they operate as expected.



The Kubelet is essential for node management in Kubernetes, acting as the intermediary between the cluster's control plane and the container runtime.

## Installing the Kubelet

Unlike other Kubernetes components, the Kubelet is not automatically deployed when you set up your cluster using tools like kubeadm. It must be installed manually on each worker node. Follow the steps below to install the Kubelet:

### Step 1: Download the Kubelet Binary

Execute the following command to download the Kubelet binary:

```
```bash theme={null} wget https://storage.googleapis.com/kubernetes-release/release/v1.13.0/bin/linux/amd64/kubelet
```

Step 2: Configure and Run the Kubelet as a Service

Set up the Kubelet with the required configuration by running it as a service. Use the command below:

```
```bash theme={null}
ExecStart=/usr/local/bin/kubelet \
 --config=/var/lib/kubelet/kubelet-config.yaml \
 --container-runtime=remote \
 --container-runtime-endpoint=unix:///var/run/containerd/containerd.sock \
 --image-pull-progress-deadline=2m \
 --kubeconfig=/var/lib/kubelet/kubeconfig \
 --network-plugin=cni \
 --register-node=true \
 --v=2
```

### Step 3: Verify the Kubelet Process

After installation, verify that the Kubelet is running by checking its process status on the worker node. Run the following command:

```
```bash theme={null} ps -aux | grep kubelet
```

This command will display all active processes containing "kubelet" along with their configuration.

```
```bash theme={null}
root 2095 1.8 2.4 960676 98788 ? Ssl 02:32 0:36 /usr/bin/kubelet --bootstrap-kub
```

Ensure that all paths and configuration files referenced in your commands exist and are correctly set up on your system.

## Next Steps

In the upcoming sections, we will explore additional Kubelet configurations, such as certificate generation and TLS bootstrapping, to further secure and optimize your Kubernetes nodes.

This concludes our lesson on the Kubelet. Continue learning in the next article to deepen your understanding of Kubernetes node management and cluster orchestration.

Happy learning!

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Kube Proxy

This article provides an overview of Kube Proxy's role in Kubernetes networking and its function in ensuring reliable communication between pods and Services.

Welcome to this detailed guide on Kube Proxy and its critical role in Kubernetes networking. In this article, you will learn how Kube Proxy ensures reliable communication between pods and how it enables Services to function seamlessly across your cluster.

## Pod Networking in Kubernetes

Kubernetes enables every pod within a cluster to communicate with one another by deploying a robust pod networking solution. This creates an internal virtual network that spans all nodes, connecting every pod.

Imagine your web application is running on one node while your database application is on another. Though the web application could connect to the database via its pod IP, these IPs are transient and may change. The recommended solution is to create a Service. By exposing the database through a Service (e.g., using the name "DB"), the web application can maintain a consistent connection without relying on fluctuating pod IPs. Each Service is assigned a stable IP address, and traffic routed to the Service is automatically forwarded to the appropriate backend pod.

A Service in Kubernetes is a virtual entity that doesn't correspond to a container or network interface. Instead, it provides a persistent endpoint in the cluster's memory, allowing stable access to the underlying pods.

## How Kube Proxy Works

Kube Proxy is a lightweight process that runs on every node in the Kubernetes cluster. Its key function is to monitor for Service creations and configure

network rules that redirect traffic to the corresponding pods. One common method it uses is by setting up IP tables rules.

For example, if a Service is assigned the IP 10.96.0.12, Kube Proxy configures the IP tables on each node so that any traffic directed to that IP is forwarded to the actual pod IP, such as 10.32.0.15. This redirection mechanism ensures that Services work transparently across the cluster, regardless of which node initiates the request.

## Installing Kube Proxy

---

To get started with Kube Proxy:

1. Download the Kube Proxy binary from the [Kubernetes release page](#).
2. Extract the binary and run it as a service on your nodes.

Typically, when using kubeadm, Kube Proxy is deployed as a DaemonSet. This approach guarantees that a single Kube Proxy pod runs on every node in the cluster.

Using kubeadm, Kube Proxy is automatically managed as a DaemonSet, streamlining the process of ensuring that every node runs a Kube Proxy instance.

## Verifying the Kube Proxy Deployment

---

After installation, you can verify that Kube Proxy is functioning correctly by checking the status of pods and the DaemonSet in the kube-system namespace. Run the following commands:

```
``bash theme={null} kubectl get pods -n kube-system
```

```
``bash theme={null}
kubectl get daemonset -n kube-system
```

These commands list the running pods and DaemonSets, confirming that Kube Proxy is deployed and operating as expected.

## Conclusion

---

This guide provided a high-level overview of Kube Proxy and its essential role in routing network traffic within Kubernetes. By understanding how Services interact with Kube Proxy, and how the component ensures reliable pod-to-pod communication, you are better prepared to manage and scale your Kubernetes deployments.

Thank you for reading. Stay tuned for more in-depth discussions on Kubernetes networking, Services, and other core components in upcoming modules.

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

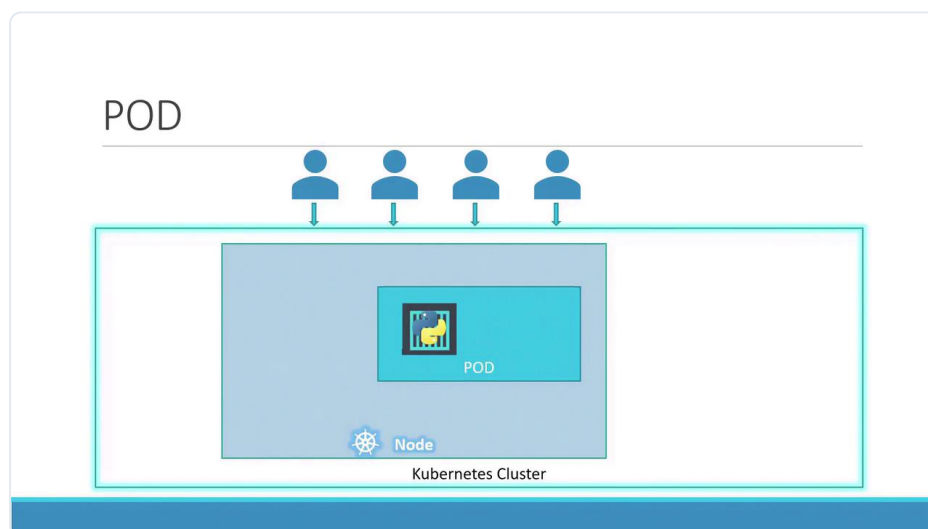
# Pods

This article provides an in-depth guide on Kubernetes Pods, covering their deployment, scaling, and management within a Kubernetes cluster.

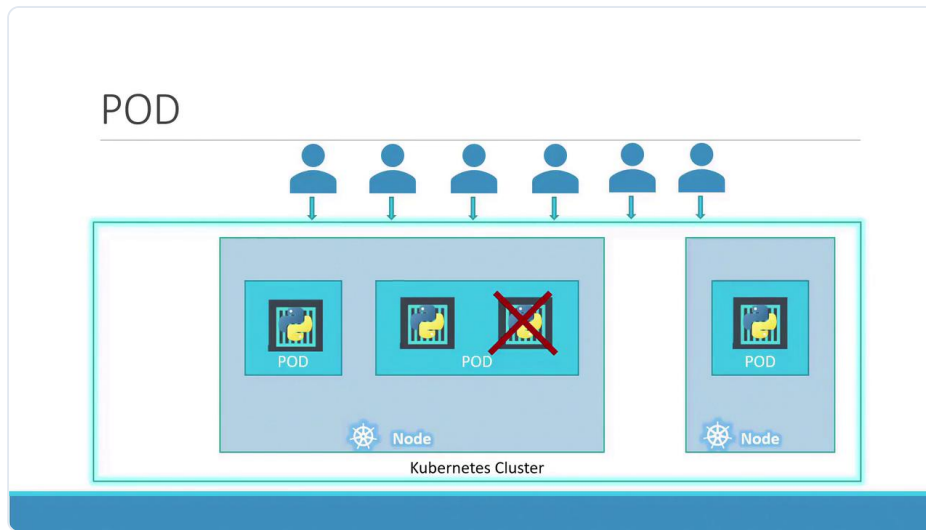
Welcome to this in-depth guide on Kubernetes Pods. In this article, we assume your application is already developed, built into Docker images, and hosted on a Docker repository (such as [Docker Hub](#)). We also assume that your Kubernetes cluster is configured and operational—whether it is a single-node or multi-node cluster. With Kubernetes, the goal is to run containers on worker nodes, but rather than deploying containers directly, Kubernetes encapsulates them within an object called a pod. A pod represents a single instance of an application and is the smallest deployable unit in Kubernetes.

In the simplest scenario, a single-node Kubernetes cluster may run one instance of your application inside a Docker container encapsulated by a pod.

When user load increases, you can scale your application by spinning up additional instances—each running in its own pod. This approach isolates each instance, allowing Kubernetes to distribute the pods across available nodes as needed.

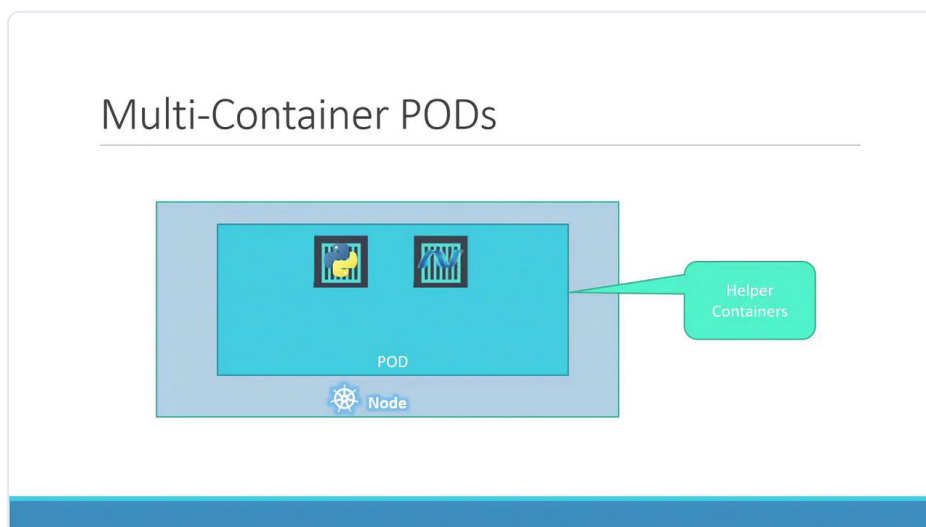


Instead of adding more containers to the same pod, additional pods are created. For instance, running two instances in separate pods allows the load to be shared across the node or even across multiple nodes if the demand escalates and additional cluster capacity is required.



Remember, scaling an application in Kubernetes involves increasing or decreasing the number of pods, not the number of containers within a single pod.

Typically, each pod hosts a single container running your main application. However, a pod can also contain multiple containers, which are usually complementary rather than redundant. For example, you might include a helper container alongside your main application container to support tasks like data processing or file uploads. Both containers in the pod share the same network namespace (allowing direct communication via localhost), storage volumes, and lifecycle events, ensuring they start and stop together.



To better understand the concept, consider a basic Docker example. Suppose you initially deploy your application with a simple command:

```
``bash theme={null} docker run python-app
```

When the load increases, you may launch additional instances manually:

```
```bash theme={null}
docker run python-app
docker run python-app
docker run python-app
docker run python-app
```

Now, if your application needs a helper container that communicates with each instance, managing links, custom networks, and shared volumes manually becomes complex. You'd have to run commands like:

```
```bash theme={null} docker run helper --link app1 docker run helper --link
app2 docker run helper --link app3 docker run helper --link app4
```

With Kubernetes pods, these challenges are resolved automatically. When a pod is defined with

Even if your current application design uses one container per pod, Kubernetes enforces the

## Deploying Pods

A common method to deploy pods is using the `kubectl run` command. For example, the following

```
```bash theme={null}
kubectl run nginx --image nginx
```

Once deployed, you can verify the pod's status with the `kubectl get pods` command. Initially, the pod might be in a "ContainerCreating" state, followed by a transition to the "Running" state as the application container becomes active. Below is an example session:

```
```bash theme={null} kubectl get pods
```

## Output:

---

NAME	READY	STATUS	RESTARTS	AGE
------	-------	--------	----------	-----

---

```
kubectl get pods
```

## Output after a few seconds:

---

NAME	READY	STATUS	RESTARTS	AGE
------	-------	--------	----------	-----

---

nginx-8586cf59-whssr	1/1	Running	0	8s
----------------------	-----	---------	---	----

---

...

At this stage, note that external access to the nginx web server has not been configured. The service is accessible only within the node. In a future article, we will explore configuring external access through Kubernetes networking and services.

After mastering pod deployment, advance to networking and service configuration to expose your applications to end users.

That concludes our discussion on Kubernetes Pods. Proceed to the demo section to see these concepts in action, and stay tuned for the next article!

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Pods with YAML

This lesson covers creating a Kubernetes Pod using a YAML file, including structure, creation, and status verification.

Welcome to this lesson on creating a Pod in Kubernetes using a YAML configuration file. In this guide, you'll learn how to structure your YAML file, create the Pod with `kubectl`, and verify its status. Kubernetes leverages YAML files to define objects such as Pods, ReplicaSets, Deployments, and Services. These definitions adhere to a consistent structure, with four essential top-level properties: `apiVersion`, `kind`, `metadata`, and `spec`.

## Top-Level Fields in a Kubernetes YAML File

Every Kubernetes definition file must include the following four fields:

```
```YAML theme={null} apiVersion: kind: metadata: spec:
```

1. **apiVersion**

This field indicates the version of the Kubernetes API you are using. For a Pod, set `apiVersion: v1`.

2. **kind**

This specifies the type of object being created. In this lesson, since we're creating a Pod, set `kind: Pod`.

3. **metadata**

The metadata section provides details about the object, including its name and labels.

```
```yaml
theme={null}
apiVersion: v1
kind: Pod
metadata:
 name: myapp-pod
 labels:
 app: myapp
spec:
  ```
```

<Callout icon="lightbulb" color="#1CB2FE">

Make sure that the properties under metadata (like name and labels) are indented to the same level. This is crucial for correct YAML parsing.

</Callout>

4. **spec**

The spec section provides specific configuration details for the object. For a Pod, this includes the containers to be run.

Below is the complete YAML configuration for our Pod:

```
```yaml
theme={null}
apiVersion: v1
kind: Pod
metadata:
 name: myapp-pod
 labels:
 app: myapp
 type: front-end
spec:
 containers:
 - name: nginx-container
 image: nginx
  ```
```

Creating and Verifying the Pod

After you have saved your configuration (for example, as `pod-definition.yaml`), use the following command to create the Pod:

```
```bash
theme={null}
kubectl create -f pod-definition.yaml
```
```

Once the Pod is created, you can verify its status by listing all Pods:

```
```bash
theme={null}
kubectl get pods
```
```

You should see output similar to this:

```
``plaintext theme={null} NAME READY STATUS RESTARTS AGE myapp-pod
1/1 Running 0 20s
```

To view detailed information about the Pod, run:

```
```bash theme={null}
kubectl describe pod myapp-pod
```

This command provides extensive details, including metadata, node assignment, container specifics, and event history such as scheduling, volume mounting, and container start-up. Here is an example output:

```
``plaintext theme={null} Name: myapp-pod Namespace: default Node:
minikube/192.168.99.100 Start Time: Sat, 03 Mar 2018 14:26:14 +0800
Labels: app=myapp Annotations: Status: Running IP: 172.17.0.24 Containers:
nginx: Container ID: docker://
830bb56c8c42a860b4b70e9c1488faelbc38663e49186bc2f5a78e7688b8c9d
Image: nginx Image ID: docker-pullable://
nginx@sha256:4771d09578c7ca65299e110b3ee1c0a2592f5ea2618d32e4ffe7a4cab1c5de
Port: State: Running Started: Sat, 03 Mar 2018 14:26:21 +0800 Ready: True
Restart Count: 0 Environment: Mounts: /var/run/secrets/kubernetes.io/
serviceaccount from default-token-x95w7 (ro)
```

Conditions: Type Status Initialized True Ready True PodScheduled True

```
Events: Type Reason Age From Message ----

Normal Scheduled 27s default-scheduler Successfully assigned myapp-pod to minikube
Normal SuccessfulMountVolume 27s minikube MountVolume.SetUp succeeded for volume "default-token-x95w7"
Normal Pulling 27s minikube pulling image "nginx"
Normal Pulled 27s minikube Successfully pulled image "nginx"
Normal Created 27s minikube Created container
Normal Started 27s minikube Started container ````
```

Using `kubectl describe` helps you gain detailed insights into the internal state of your Pod, which can be invaluable for troubleshooting.

## Conclusion

In this lesson, you learned how to structure a Kubernetes YAML file for a Pod, create it using `kubectl`, and verify its status. This hands-on approach equips you to manage and troubleshoot your Kubernetes resources effectively. Happy Kubernetes-ing!

For more information, refer to the following resources:

- [Kubernetes Documentation](#)
- [Kubernetes Basics](#)



## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Demo Pods with YAML

This article demonstrates how to create a Kubernetes Pod using a YAML definition file for better control over pod specifications.

In this lesson, we will create a Kubernetes Pod using a YAML definition file instead of the "kubectl run" command. This method offers more control by allowing you to define pod specifications explicitly in a file. You can choose any text editor for this task; for instance, Windows users may prefer Notepad++ over Notepad, while Linux users might opt for vim. In future sections, we will explore additional IDEs and tools to streamline YAML editing, but we will stick with the basics for now.

## Step 1: Creating the YAML File

Open your terminal and use vim to create a file named pod.yaml:

```
```bash theme={null} vim pod.yaml
```

Inside the file, define the following key elements:

- * **apiVersion:** Should be set to `v1` for a Pod.
- * **kind:** Must be `Pod` (case-sensitive).
- * **metadata:** A dictionary that includes the pod's name and any labels used for grouping.
- * **spec:** Contains the pod specifications, including a list of containers.

<Callout icon="lightbulb" color="#1CB2FE">

Be sure to follow proper indentation rules. Use two spaces per level (avoid tabs), as misalignment can lead to errors.

</Callout>

Below is a complete example configuration for a single-container Pod using the `nginx` image.

```
```yaml theme={null}
apiVersion: v1
kind: Pod
metadata:
 name: nginx
 labels:
 app: nginx
 tier: frontend
spec:
 containers:
 - name: nginx
 image: nginx
```

To add additional containers, insert another block within the containers list with the appropriate name and image.

## Step 2: Saving and Verifying the YAML File

After editing the file, exit vim and save your changes by typing:

```
```bash theme={null} :wq
```

Verify the contents of your YAML file with:

```
```bash theme={null}
cat pod.yaml
```

The output should match the YAML configuration shown above.

## Step 3: Creating the Pod in the Cluster

Create the Pod on your Kubernetes cluster using your YAML file. You can use either the `kubectl create` or `kubectl apply` command. Here's an example with `kubectl apply`:

```
```bash theme={null} kubectl apply -f pod.yaml
```

Output:

pod/nginx created

To check the status of your Pod, run:

```
```bash theme={null}
kubectl get pods
```

Initially, you might see an output similar to this:

```
```bash theme={null} NAME READY STATUS RESTARTS AGE nginx 0/1
ContainerCreating 0 7s
```

After a short while, re-running the command should show the Pod in a running state:

```
```bash theme={null}
kubectl get pods
```

Example output:

```
```bash theme={null} NAME READY STATUS RESTARTS AGE nginx 1/1
Running 0 9s
```

Step 4: Inspecting the Pod Details

For a detailed overview of your Pod, use the `kubectl describe` command:

```
```bash theme={null}
kubectl describe pod nginx
```

This command provides comprehensive details about the Pod, including container statuses, event logs, volumes, and node assignments. Below is an example of typical output:

```
bash theme={null} Initialized True Ready True ContainersReady True
PodScheduled True Volumes: default-token-f5ntk: Type: Secret (a
volume populated by a Secret) SecretName: default-token-f5ntk
Optional: false QoS Class: BestEffort Node-Selectors: <none>
Tolerations: node.kubernetes.io/not-ready:NoExecute for 300s
node.kubernetes.io/unreachable:NoExecute for 300s Events: Type
Reason Age From Message ---- -
Scheduled 21s default-scheduler Successfully assigned default/nginx
to minikube Normal Pulling 20s kubelet, minikube Pulling image
"nginx" Normal Pulled 14s kubelet, minikube Successfully pulled
```

```
image "nginx" Normal Created 14s kubelet, minikube Created container
nginx Normal Started 14s kubelet, minikube Started container nginx
```

## Conclusion

---

This demonstration has guided you through creating a Kubernetes Pod using a YAML configuration file. This approach not only reinforces good configuration practices but also provides enhanced flexibility compared to command-based object creation. In our next lesson, we will cover advanced IDEs and tools to further ease YAML file management.

For additional reading and resources, check out:

- [Kubernetes Documentation](#)
- [Kubernetes Basics](#)

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Practice Test Introduction

This article introduces a practice test designed to help users prepare for an exam in a realistic lab environment.

Before you begin the practice test, take a moment to familiarize yourself with the practice test portal. Once the lecture is completed, you will be directed to the lab environment where you'll complete various hands-on exercises.

This practice test is designed to help you prepare for the exam by testing your skills in a realistic lab setting. Please use a laptop or desktop computer with a keyboard for the best experience.

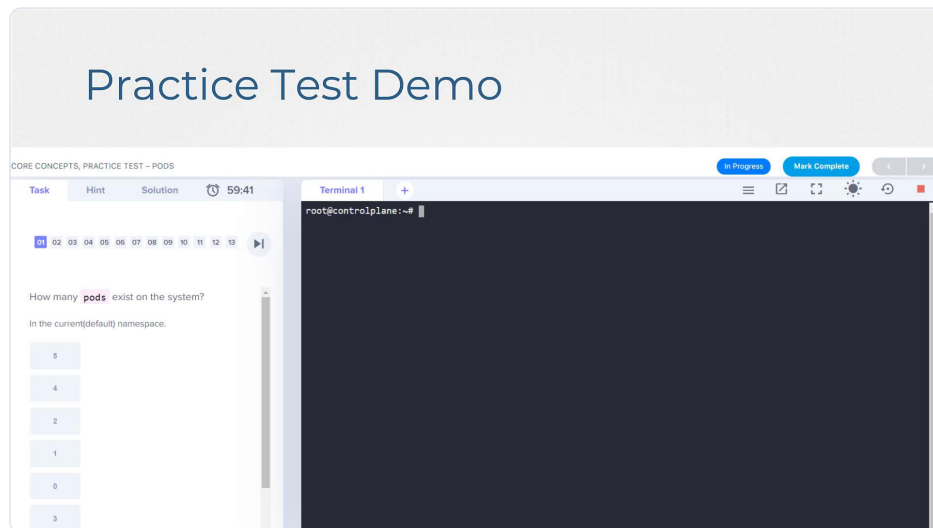
## Lab Environment Overview

Click the start button to load the lab environment. Allow a moment for the lab to initialize—typically less than 30 seconds, though occasionally it may take a few minutes.

Once loaded, you will see two main sections:

- **Quiz Portal (Left Side):** Hosts your test questions.
- **Live Terminal (Right Side):** Where you execute commands. This terminal might be a Linux shell for Linux, shell scripting, or Git tasks; a Docker host for Docker tasks; or a Kubernetes control plane for Kubernetes tasks. You can run commands here to interact with and configure the system.

Below is an image illustrating the practice test interface:



## Structure of the Practice Test

The lab is divided into two parts:

1. **Terminal:** Use this section on the right to run commands and interact with the system (e.g., view and configure a Kubernetes cluster).
2. **Quiz Portal:** Located on the left, this portal contains practice test questions grouped by topic. Each practice test includes between 5 to 15 questions. The total number of questions is shown at the top, and each question is marked as you progress.

## Question Types

The test includes a variety of question types for a comprehensive learning experience:

- **Environment-Based Questions:** Find information within the lab environment to select the correct answer.
- **Configuration Tasks:** Modify the environment by deploying Pods, services, or other components, then verify the changes.
- **Specification Matching:** Given a configuration specification (spec), create a matching configuration using the provided editors (e.g., the vi editor).
- **Multiple-Choice Questions:** Some questions are scenario-based to test your conceptual understanding. Note that these might not appear on the actual certification exam.

If a configuration does not exactly match the given specification, adjust your settings precisely. You can skip questions by clicking the skip button at the top; however, once skipped, these questions cannot be revisited due to the fixed question sequence.

## Accessing Web Applications

For certain deployments, you might have a web portal link available above the terminal. This link is provided if the application features a web interface. For instance, when exposing a service, you might run the following command:

```
bash theme={null} root@controlplane:~# kubectl expose pod nginx --port 80 --type NodePort service/nginx exposed root@controlplane:~#
```

If you need an additional terminal window, use the designated button at the top.

Please note that this lab environment is not an exact replica of the actual Kubernetes certification exam environment. It is custom-built to help you practice and prepare. All work will be deleted after one hour (or the specified lab session duration), so do not store any personal credentials or persistent data here.

## Retaking the Test and Session Guidelines

You can retake these tests as many times as needed until you feel confident. However, remember:

- The lab environment is temporary and will expire after one hour (or as specified).
- Refreshing the lab window or experiencing temporary connectivity issues should return you to the same environment.
- Extended browser closures or lengthy idle periods may result in the lab being automatically terminated. In such cases, you will need to restart a new lab session from the beginning.

Below is an image that reminds you of the temporary nature of the lab environment:

### Please Note

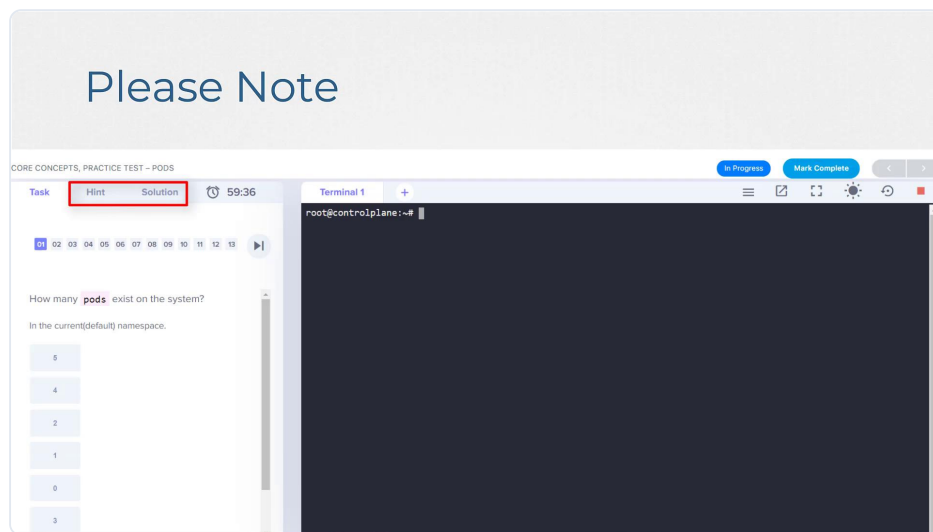
- This is **not** the interface of the actual exam
- As many times as you like
- Temporary Environment – Only available for **1 Hour**
- **No** Personal details

## Practice and Learning

These practice tests are designed not only to assess your skills based on the course lecture but also to help you gain essential hands-on experience. Some questions may cover topics beyond the lecture material, so always refer to the documentation and search for solutions if you are stuck. The hints and solutions panel, visible with each question, is provided to guide you.

If any task or question remains unclear, please reach out for further assistance. The goal is to encourage you to read the documentation, find necessary information, and apply it successfully.

Below is an additional image showing a Kubernetes practice test interface:



For further learning, consider exploring:

- [Kubernetes Documentation](#)
- [Kubernetes Basics](#)
- [Docker Hub](#)

Good luck with your practice test and exam preparation!

## Documentation Index

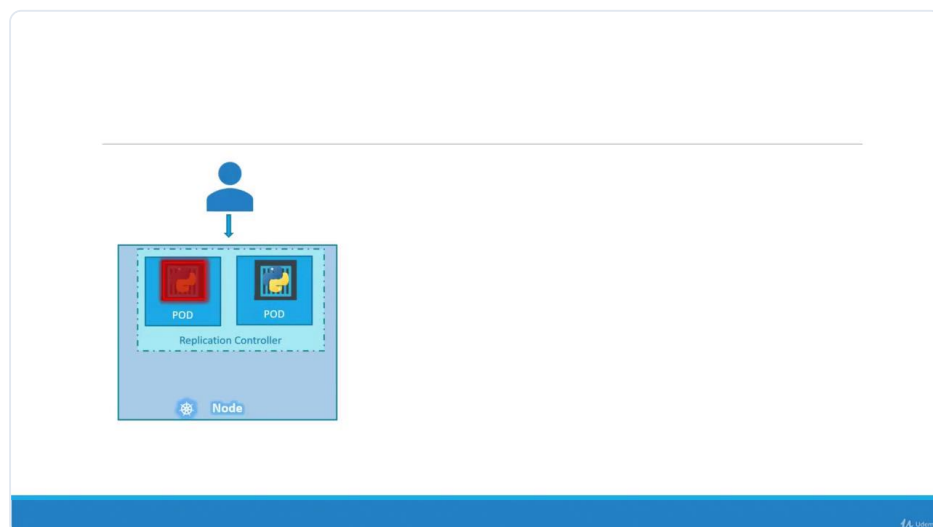
Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# ReplicaSets

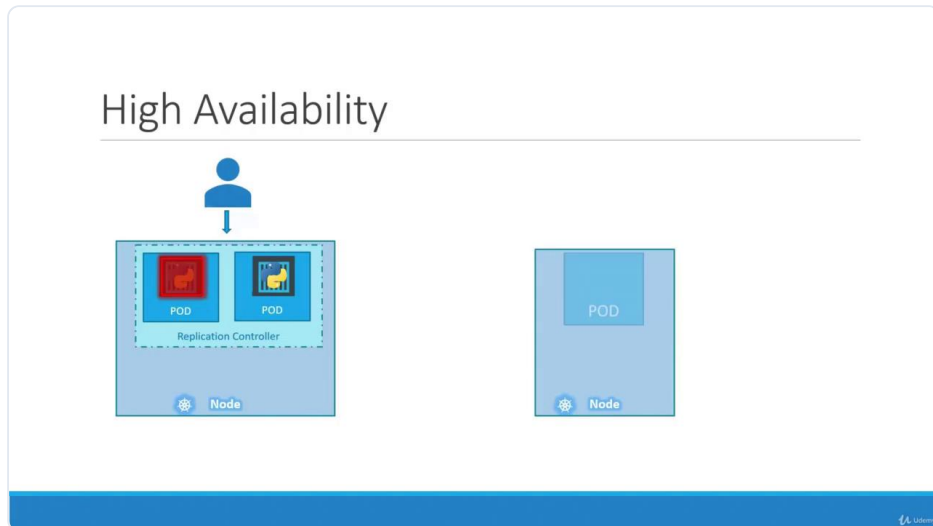
This article explains Kubernetes replication controllers and ReplicaSets, focusing on their roles in maintaining high availability and load balancing in clusters.

Hello, and welcome to this lesson on Kubernetes controllers. I'm Mumshad Mannambeth, and today we'll dive into the essential components that drive Kubernetes operations. Kubernetes controllers continuously monitor objects and take necessary actions, and in this lesson, we focus on the replication controller—an essential building block for maintaining high availability in your cluster.

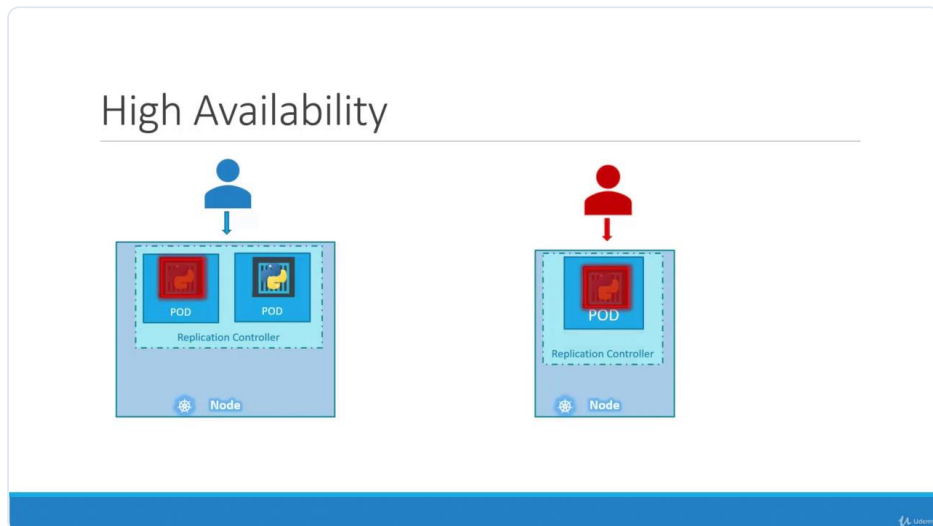
Imagine a scenario where a single pod runs your application. If that pod crashes or fails, users lose access. To prevent this risk, running multiple pod instances is key. A replication controller ensures high availability by creating and maintaining the desired number of pod replicas. Even if you intend to run a single pod, a replication controller adds redundancy by automatically creating a replacement if the pod fails.



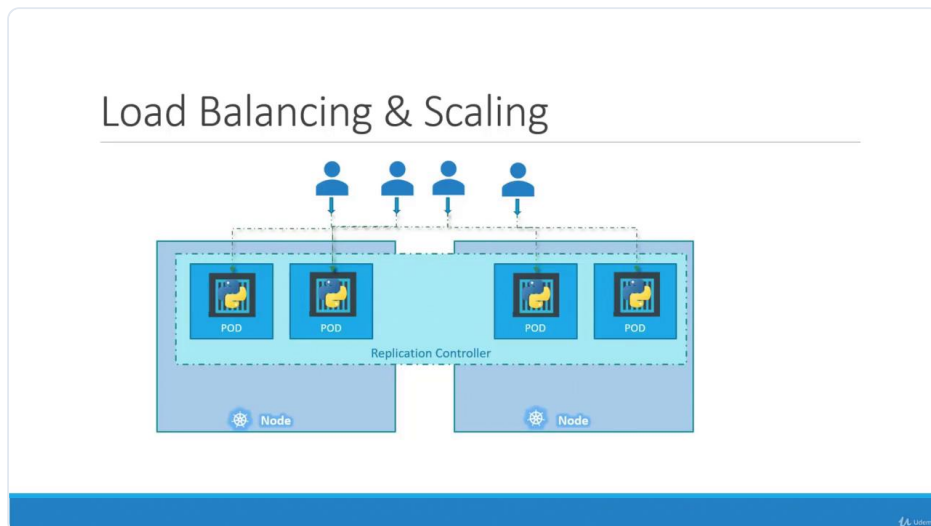
If one pod serving your application crashes, the replication controller immediately deploys a new one to keep the service available.



For example, if you need to maintain a constant service level, the controller ensures the desired number of pods—whether one or one hundred—are always running.



Beyond availability, replication controllers also help distribute load. When user demand increases, additional pods can better balance that load. If resources on a particular node become scarce, new pods can be scheduled across other nodes in your cluster.



While both replication controllers and replica sets serve similar purposes, the replication controller is the older technology being gradually replaced by the replica set. In this lesson, we will focus on replica sets for our demos and implementations.

## Creating a Replication Controller

To create a replication controller, start by writing a configuration file (e.g., `rc-definition.yaml`). Like any Kubernetes manifest, the file contains four main sections: `apiVersion`, `kind`, `metadata`, and `spec`.

1. **apiVersion:** For a replication controller, use `v1`.
2. **kind:** Set this to `ReplicationController`.
3. **metadata:** Provide a name (e.g., `myapp-rc`) and include labels such as `app` and `type`.
4. **spec:** This section is crucial. It not only defines the desired number of replicas with the `replicas` key but also includes a `template` section which serves as the blueprint for creating the pods. Ensure that all pod-related entries in the template are indented correctly and aligned with `replicas` as siblings.

Once your YAML file is ready, create the replication controller using the following command:

```
``bash theme={null} kubectl create -f rc-definition.yml
```

Below is a complete example of a replication controller definition:

```
```yaml theme={null}
apiVersion: v1
kind: ReplicationController
metadata:
  name: myapp-rc
  labels:
    app: myapp
    type: front-end
spec:
  replicas: 3
  template:
    metadata:
      name: myapp-pod
      labels:
        app: myapp
        type: front-end
    spec:
      containers:
        - name: nginx-container
          image: nginx
```

When you run the following command, Kubernetes creates three pods according to the provided template:

```
```bash theme={null} kubectl create -f rc-definition.yml
```

## Output:

---

# replicationcontroller "myapp-rc" created

---

To view the replication controller and its pods, run these commands

```
```bash theme={null}
kubectl get replicationcontroller
kubectl get pods
```

A sample output might look like:

```
```bash theme={null}
```

```
kubectl get replicationcontroller NAME DESIRED CURRENT READY
AGE myapp-rc 3 3 3 19s
```

```
kubectl get pods NAME READY STATUS RESTARTS AGE myapp-rc-4lvk9 1/1 Running 0 20s myapp-rc-mc2mf 1/1 Running 0 20s myapp-rc-px9pz 1/1 Running 0 20s
```

Notice that the pods' names include the replication controller's name ("myapp-rc"), indicating

\*\*\*

## ## Introducing ReplicaSet

A ReplicaSet is a modern alternative to the replication controller, using an updated API version.

1. **API Version**: Use `apps/v1` for a ReplicaSet.
2. **Selector**: In addition to metadata and specification, a ReplicaSet requires a `selector` field.

Below is an example ReplicaSet definition:

```
```yaml theme={null}
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: myapp-replicaset
  labels:
    app: myapp
    type: front-end
spec:
  replicas: 3
  selector:
    matchLabels:
      type: front-end
  template:
    metadata:
      name: myapp-pod
      labels:
        app: myapp
        type: front-end
    spec:
      containers:
        - name: nginx-container
          image: nginx
```

Create the ReplicaSet with:

```
```bash theme={null} kubectl create -f replicaset-definition.yml
```

Then, verify its creation:

```
```bash theme={null}
kubectl get replicaset
```

And view the associated pods:

```
```bash theme={null} kubectl get pods
```

```

```

```
Labels and Selectors
```

Labels in Kubernetes are critical because they enable controllers, such as ReplicaSets, to

```
```yaml theme={null}
```

```
selector:
```

```
  matchLabels:
```

```
    tier: front-end
```

The pod definition should similarly include the label:

```
yaml theme={null} metadata: name: myapp-pod labels: tier: front-end
```

This label-selector mechanism ensures that the ReplicaSet precisely targets the intended pods and maintains the set number of replicas by replacing any failed pods.

Is the Template Section Required?

Even if three pods with matching labels already exist in your cluster, the template section in the ReplicaSet specification remains essential. It serves as the blueprint for creating new pods if any fail, ensuring the desired state is consistently maintained.

Scaling the ReplicaSet

Scaling a ReplicaSet involves adjusting the number of pod replicas. There are two methods to achieve this:

1. Update the Definition File

Modify the `replicas` value in your YAML file (e.g., change from 3 to 6) and update the ReplicaSet with:

```
bash theme={null} kubectl replace -f replicaset-definition.yml
```

1. Use the `kubectl scale` Command

Scale directly from the command line:

```
bash theme={null} kubectl scale --replicas=6 -f replicaset-definition.yml
```

Keep in mind that if you scale using the `kubectl scale` command, the YAML file still reflects the original number of replicas. To maintain consistency, it may be necessary to update the YAML file after scaling.

Common Commands Overview

Below is a quick reference table summarizing some useful commands when working with replication controllers and ReplicaSets:

Resource Type	Use Case	Example Command
Create Object	Create from a definition file	<code>kubectl create -f <filename></code>
View ReplicaSets/RC	List replication controllers	<code>kubectl get replicaset</code> or <code>kubectl get replicationcontroller</code>
Delete ReplicaSet/RC	Remove a replication controller	<code>kubectl delete replicaset <replicaset-name></code>
Update Definition	Replace object using YAML file	<code>kubectl replace -f <filename></code>
Scale ReplicaSet/RC	Change number of replicas	<code>kubectl scale --replicas=<number> -f <filename></code>

That concludes our lesson on ReplicaSets and replication controllers in Kubernetes. Understanding these concepts is vital for managing high availability and load balancing in your cluster. Happy learning!

For further reading, check out [Kubernetes Documentation](#).

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Deployments

This guide explores Kubernetes deployments, simplifying application management with features like rolling updates, rollbacks, and high availability.

Hello and welcome! My name is Mumshad Mannambeth. In this guide, we dive into Kubernetes deployments—an abstraction that simplifies managing your applications in a production environment. Rather than interacting directly with pods and ReplicaSets, deployments offer advanced features that enable you to:

- Deploy multiple instances of your application (like a web server) to ensure high availability and load balancing.
- Seamlessly perform rolling updates for Docker images so that instances update gradually, reducing downtime.
- Quickly roll back to a previous version if an upgrade fails unexpectedly.
- Pause and resume deployments, allowing you to implement coordinated changes such as scaling, version updates, or resource modifications.

Previously, we discussed how individual pods encapsulate containers and how ReplicaSets maintain multiple pod copies. A deployment, however, sits at a higher level, automatically managing ReplicaSets and pods while providing enhanced features like rolling updates and rollbacks.

Creating a Deployment

To create a deployment, start by writing a deployment definition file. This file is similar to a ReplicaSet definition, with the key difference being that the kind is set to Deployment instead of ReplicaSet. Below is an example of a correct deployment definition file:

```
```yaml
theme={null}
apiVersion: apps/v1
kind: Deployment
metadata:
 name: myapp-deployment
 labels:
 app: myapp
 type: front-end
spec:
 replicas: 3
```

```
selector: matchLabels: type: front-end template: metadata: labels: app: myapp
type: front-end spec: containers: - name: nginx-container image: nginx
```

```
Once your deployment definition file (for example, named deployment-definition.yml) is ready,
you can create the deployment by running the following command:

```bash theme={null}
kubectl create -f deployment-definition.yml
```

The command output should confirm that the deployment has been created:

```
```console theme={null} deployment "myapp-deployment" created
```

To verify the deployment, run:

```
```bash theme={null}
kubectl get deployments
```

The output will look similar to this:

```
```console theme={null} NAME DESIRED CURRENT UP-TO-DATE
AVAILABLE AGE myapp-deployment 3 3 3 21s
```

```
Behind the Scenes: How Deployments Work
```

When you create a deployment, Kubernetes automatically creates an associated ReplicaSet. To

```
```bash theme={null}
kubectl get replicaset
```

You'll notice a new ReplicaSet with a name derived from your deployment. This ReplicaSet oversees the creation and management of pods. To view the pods managed by the ReplicaSet, run:

```
```bash theme={null} kubectl get pods
```

While deployments and ReplicaSets work together seamlessly, deployments provide additional

<Callout icon="lightbulb" color="#1CB2FE">

To view all the created Kubernetes objects—deployments, ReplicaSets, pods, and more—use t

```
```bash theme={null}
kubectl get all
```
```

This gives you a comprehensive overview of your deployment's components.

</Callout>

A sample output of the "kubectl get all" command might be:

```
```console theme={null}
NAME                                DESIRED  CURRENT  UP-TO-DATE  AVAILABLE  AGE
deploy/myapp-deployment             3        3        3           3          9h

NAME                                DESIRED  CURRENT  READY  AGE
rs/myapp-deployment-6795844b58      3        3        3      9h

NAME                                READY  STATUS   RESTARTS  AGE
po/myapp-deployment-6795844b58-5rbjl 1/1    Running  0         9h
po/myapp-deployment-6795844b58-h4w55 1/1    Running  0         9h
po/myapp-deployment-6795844b58-1fjhv 1/1    Running  0         9h
```

In this output, you can clearly see the deployment, its associated ReplicaSet, and the managed pods.

Conclusion

This article has covered the fundamentals of creating a deployment in Kubernetes. By leveraging deployments, you gain powerful capabilities like rolling updates and rollbacks that make managing application updates and maintenance in production more efficient. Whether you are scaling your application or rolling out new features, Kubernetes deployments provide a robust solution for modern application management.

Happy deploying!

Documentation Index

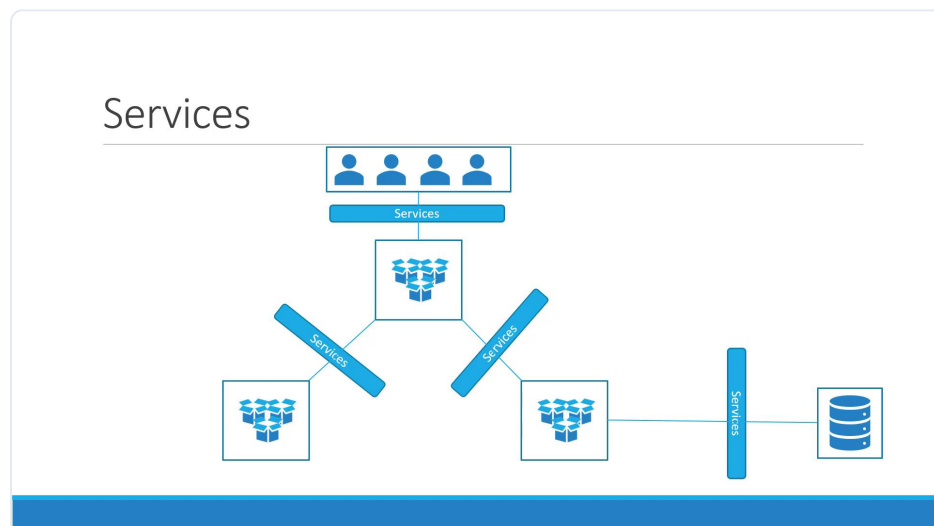
Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Services

This guide explores Kubernetes services, focusing on enabling communication between application components and detailing the creation and use of NodePort services.

Hello, and welcome to this in-depth guide on Kubernetes Services for Beginners. My name is Mumshad Mannambeth, and in this article, we'll explore how Kubernetes services enable seamless communication between various application components—both within the cluster and from the outside world.

Kubernetes services allow different sets of Pods to interact with each other. Whether connecting the front end to back-end processes or integrating external data sources, services help to decouple microservices while maintaining reliable communication. For instance, you can expose your front-end to end users and enable back-end components to interact efficiently.



Use Case: From Internal Networking to External Access

So far, we've seen how Pods communicate internally using the Kubernetes network. Consider a scenario where you deploy a Pod running a web application and want an external user to access it. Here's a quick overview of the setup:

- **Kubernetes Node IP:** 192.168.1.2
- **Laptop IP (same network):** 192.168.1.10
- **Internal Pod Network:** 10.244.0.0
- **Pod IP:** 10.244.0.2

Since the Pod is on an isolated internal network, direct access to 10.244.0.2 from your laptop isn't possible. One workaround is to SSH into the Kubernetes node (192.168.1.2) and use `curl` to reach the Pod:

```
``bash theme={null} curl http://10.244.0.2 Hello World!
```

While this method works from the node, the goal is to have external access directly from your laptop.

For example:

```
``bash theme={null}
curl http://192.168.1.2:30008
Hello World!
```

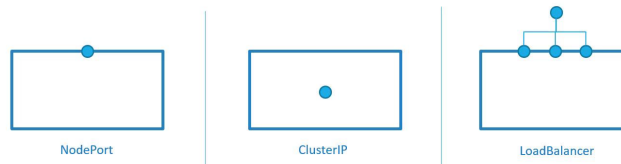
This configuration externally exposes the web server running inside the Pod.

Types of Kubernetes Services

Kubernetes supports several service types, each serving a unique purpose:

- **NodePort:** Maps a port on the node to a port on a Pod.
- **ClusterIP:** Creates a virtual IP for internal communication between services (e.g., connecting front-end to back-end servers).
- **LoadBalancer:** Provisions an external load balancer (supported in cloud environments) to distribute traffic across multiple Pods.

Services Types



Remember: The NodePort service type maps a specific node port (e.g., 30008) to the target port on your Pod (e.g., 80). This provides external access while keeping internal port targeting intact.

NodePort Service Breakdown

With a NodePort service, there are three key ports to consider:

1. **Target Port:** The port on the Pod where the application listens (e.g., 80).
2. **Port:** The virtual port on the service within the cluster.
3. **NodePort:** The external port on the Kubernetes node (by default in the range 30000–32767).

Creating a NodePort Service

The process of creating a NodePort service begins with defining the service in a YAML file. The definition file follows a similar structure to those used for Deployments or ReplicaSets, including API version, kind, metadata, and spec.

Below is an example YAML file that defines a NodePort service:

```
``yaml theme={null} apiVersion: v1 kind: Service metadata: name: myapp-service spec: type: NodePort ports: - targetPort: 80 port: 80 nodePort: 30008
```

In this YAML:

- * `targetPort` specifies the Pod's application port.
- * `port` is the port on the service that acts as a virtual server port within the cluster.
- * `nodePort` maps the external request to the specific port on the node (ensure it's between 30000 and 32767).

Note that if you omit `targetPort`, it defaults to the same value as `port`. Similarly, if you omit `nodePort`, it defaults to a random port.

However, this YAML definition does not link the service to any Pods. To connect the service to the Pods, you need to define a selector.

```
```yaml
theme: {null}
apiVersion: v1
kind: Pod
metadata:
 name: myapp-pod
 labels:
 app: myapp
 type: front-end
spec:
 containers:
 - name: nginx-container
 image: nginx
```
```

Now, update the service definition to include a selector that matches these labels:

```
```yaml
theme: {null}
apiVersion: v1
kind: Service
metadata:
 name: myapp-service
spec:
 type: NodePort
 ports:
 - targetPort: 80
 port: 80
 nodePort: 30008
 selector:
 app: myapp
 type: front-end
```
```

Save the file as `service-definition.yml` and create the service using:

```
```bash
theme: {null}
kubectl create -f service-definition.yml
```
```

You should see a confirmation message:

```
```bash
theme: {null}
service "myapp-service" created
```
```

Verify the service details with:

```
```bash
theme: {null}
kubectl get services
```
```

An example output might be:

```
```bash
theme: {null}
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S) AGE
kubernetes ClusterIP 10.96.0.1 443/TCP 16d
myapp-service NodePort 10.106.127.123 80:30008/TCP 5m
```
```

```
Access the web service externally by pointing your browser or using curl with the node IP

```bash theme={null}
curl http://192.168.1.2:30008
```

A typical response from an Nginx server might be:

```
```html theme={null}
```

... `` ## Kubernetes Services in Production In a production environment, your application is likely spread across multiple Pods for high availability and load balancing. When Pods share matching labels, the service automatically detects and routes traffic to all endpoints. Kubernetes employs a round-robin (or random) algorithm to distribute incoming requests, serving as an integrated load balancer. Furthermore, even if your Pods are spread across multiple nodes, Kubernetes ensures that the target port is mapped on all nodes. This means you can access your web application using the IP of any node along with the designated NodePort, providing reliable external connectivity. [The image illustrates a Kubernetes NodePort service setup, showing nodes with IPs, ports, and pods, enabling external access to services within a cluster.] (https://kodekloud.com/kk-media/image/upload/v1752869747/notes-assets/images/CKA-Certification-Course-Certified-Kubernetes-Administrator-Services/frame_750.jpg) Regardless of whether your application runs on a single Pod on one node, multiple Pods on a single node, or Pods spread across several nodes, the service creation process remains consistent. Kubernetes automatically updates the service endpoints when Pods are added or removed, ensuring a flexible and scalable infrastructure. ## Summary This article has provided a comprehensive introduction to Kubernetes NodePort services, covering the following key points: * The purpose and importance of Kubernetes services in enabling both internal and external communications. * A detailed explanation of how NodePort services work and the roles of targetPort, service port, and nodePort. * Step-by-step instructions on creating a NodePort service and linking it to your Pods via selectors. * An overview of production scenarios where multiple Pods ensure high availability and load balancing. Thank you for reading this guide on Kubernetes NodePort services. Explore the demo, and I look forward to sharing more Kubernetes insights in future articles. For further reading, check out: * [Kubernetes Documentation] (<https://kubernetes.io/docs/>) * [Kubernetes Basics] (<https://kubernetes.io/docs/concepts/overview/what-is-kubernetes/>) * [Docker Hub] (<https://hub.docker.com/>)

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Services Cluster IP

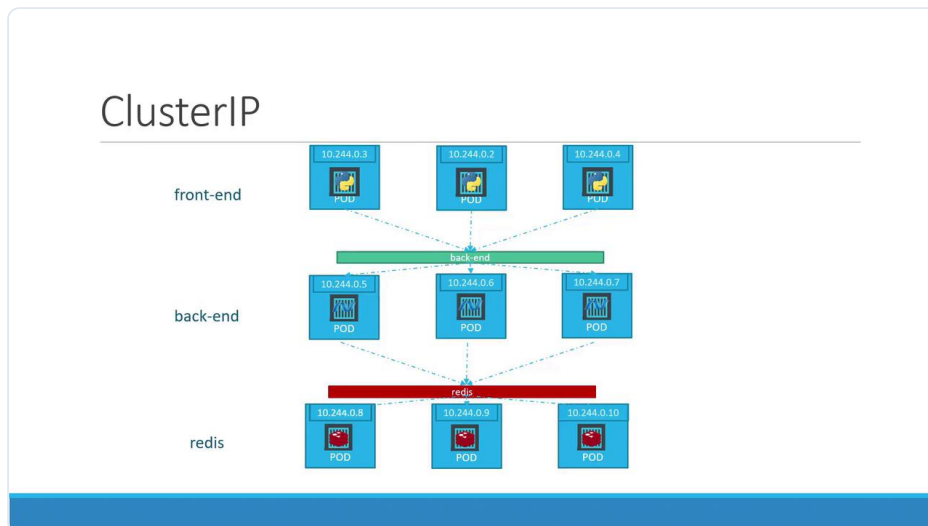
This article explains how Kubernetes Service Cluster IP facilitates stable pod-to-pod communication in microservices-based applications.

Welcome to this lesson on Kubernetes Service Cluster IP. In this guide, we explain how Cluster IP streamlines connectivity within a full-stack web application by providing a stable interface for pod-to-pod communication.

A typical microservices-based application consists of several pods. Some pods host a front-end web server, while others run a back-end server; additional pods manage services like a key-value store using Redis or persistent databases like MySQL. The front-end pods need to communicate with the back-end services, and the back-end servers must interact with databases and caching mechanisms.

Because pods receive dynamic IP addresses that can change when they are recreated, relying on these IPs for internal communication is impractical. Moreover, when a front-end pod (for example, with IP 10.244.0.3) needs to connect to a back-end service, there arises the issue of determining which pod should handle the request. Kubernetes solves this challenge by grouping related pods under a single service. This service provides a fixed Cluster IP or a service name, allowing other pods to access them without worrying about individual IPs. The service automatically load-balances incoming requests among the available pods.

For instance, by creating a service for the back-end pods, you can group them together under one interface. Similarly, services can be set up for Redis or other application tiers, ensuring that each layer can scale independently without disrupting internal connectivity.



Each service in Kubernetes is automatically assigned an IP and DNS name within the cluster. This Cluster IP should be used by other pods when accessing the service, ensuring consistent and reliable connectivity.

Example: "back-end" Service

Below is a sample YAML configuration for creating a service named "back-end". This service exposes port 80 on the Cluster IP, forwarding requests to the back-end pods that match the specified labels (`app: myapp` and `type: back-end`). The `targetPort` is set to 80, matching the port where the back-end container listens:

```
```yaml
theme={null}
apiVersion: v1
kind: Service
metadata:
 name: back-end
spec:
 type: ClusterIP
 ports:
 - port: 80
 targetPort: 80
 selector:
 app: myapp
 type: back-end
```
```

To create the service, run the following command:

```
```bash
theme={null}
kubectl create -f service-definition.yml
```
```

After deploying the service, verify its status with:

```
```bash
theme={null}
kubectl get services
```
```

The output should resemble the following:

```
```plaintext
theme={null}
NAME TYPE CLUSTER-IP EXTERNAL-IP PORT(S)
kubernetes ClusterIP 10.96.0.1 <none> 443/TCP
back-end ClusterIP 10.106.127.123 <none> 80/TCP
```
```

With this setup, components of your application can access the back-end service using either its Cluster IP or its DNS service name, ensuring uninterrupted connectivity even as individual pods scale dynamically.

This concludes the lesson. Thank you for reading, and we look forward to seeing you in the next lesson.

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Services Loadbalancer

This article explores the LoadBalancer service type in Kubernetes, focusing on simplifying access to front-end applications through cloud-native load balancing.

In this article, we explore another service type: the LoadBalancer. Previously, we examined the NodePort service that exposes an application on a specific port of the worker nodes. Here, we focus on front-end applications such as the voting app and the result app.

Imagine that these pods are distributed across a cluster—say, a four-node cluster. To allow external users to access these applications, services of type NodePort were created. The NodePort service routes incoming traffic from designated ports on the worker nodes to the corresponding pods. With NodePort, you can reach the applications using any node's IP address along with its high port number. For example, if the voting app and the result app are bound to different IP-port combinations, users could access the application using any node's IP address and its specified port, even if the pods are running on only two of the nodes.

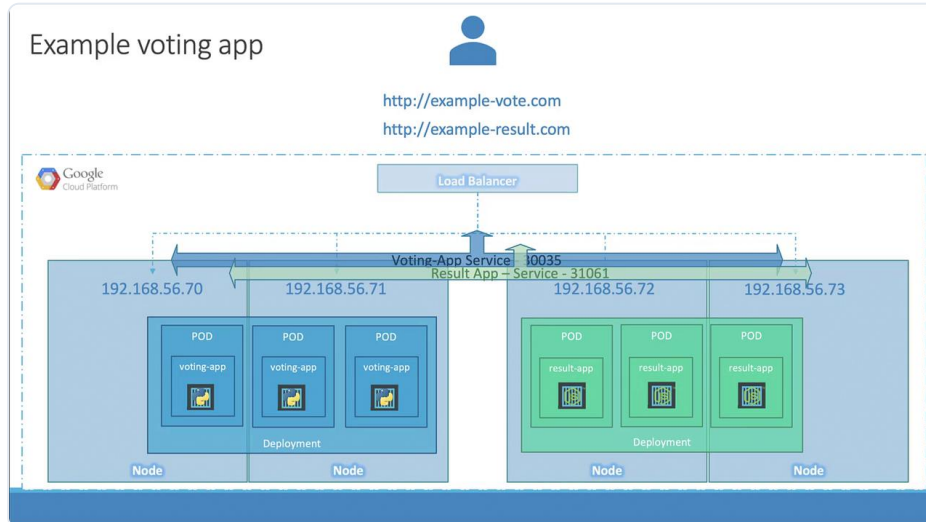
While NodePort works, it forces users to remember multiple IP-port pairs, which can be inconvenient.

End users typically prefer a single URL—for instance, `votingapp.com` or `resultapp.com`—rather than juggling several combinations. One approach to provide a unified entry point is to set up a dedicated virtual machine as a load balancer, using tools like [HAProxy](#) or [Nginx](#) to distribute the traffic across your nodes. However, managing an external load balancer can add complexity and overhead.

For a simpler solution, many cloud platforms such as [Google Cloud Platform \(GCP\)](#), [AWS](#), or [Azure](#) offer integrated load balancing. When running Kubernetes on one of these supported cloud environments, you can simply change the service type from NodePort to LoadBalancer for your front-end services. Kubernetes then automatically provisions and configures a cloud-

native load balancer, providing a single, user-friendly URL to access your application.

Below is a diagram that illustrates a cloud-based voting app architecture on Google Cloud Platform. The image shows nodes, pods, deployments, and a load balancer configured to handle the voting and result services:



Keep in mind that the LoadBalancer service type only functions as intended on supported cloud environments. In unsupported settings—such as VirtualBox—the LoadBalancer type behaves like NodePort by exposing the service on a high port without providing external load balancing.

In upcoming articles, we will demonstrate how to deploy applications on cloud platforms and observe how this configuration works in action. Stay tuned for more detailed guides in our series.

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

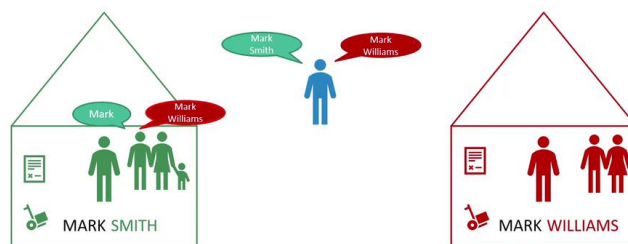
Namespaces

This article explores Kubernetes namespaces, their role in organizing resources, and how to manage them effectively within a cluster.

In this lesson, we explore the concept of namespaces in Kubernetes and how they help organize and isolate resources within your cluster.

Understanding Namespaces Through Analogy

Imagine there are two boys named Mark. To differentiate between them, you refer to them by their last names—Smith and Williams. They come from different houses where people often use first names for those familiar with them. However, when addressing someone from another house or an outsider, the full name is used. In Kubernetes, these "houses" represent namespaces. They allow you to group and manage resources differently based on their context and intended use.

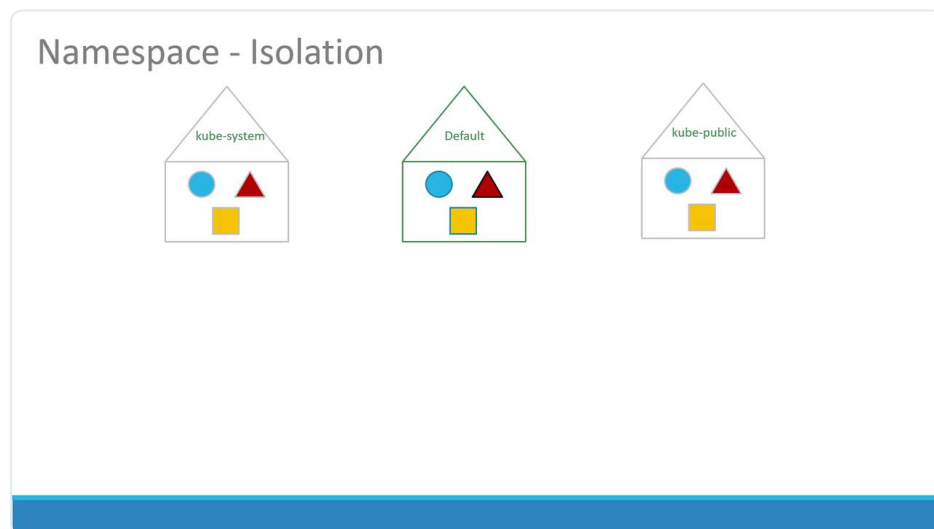


Default Namespace and System Namespaces

By default, when you create objects such as pods, deployments, and services in your cluster, they are placed within a specific namespace (similar to being "inside a house"). The default namespace is automatically created during the Kubernetes cluster setup. Additionally, several system namespaces are created at startup:

- **kube-system:** Contains core system components like network services and DNS, segregated from user operations to prevent accidental changes.
- **kube-public:** Intended for resources that need to be publicly accessible across users.

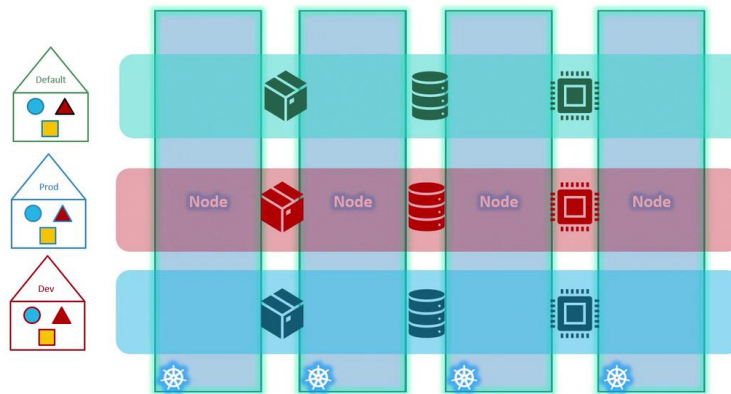
If you're running a small environment or a personal cluster for learning, you might predominantly use the default namespace. In enterprise or production environments, however, namespaces provide essential isolation and resource management by allowing environments like development and production to coexist on the same cluster.



Isolating Resources with Namespaces

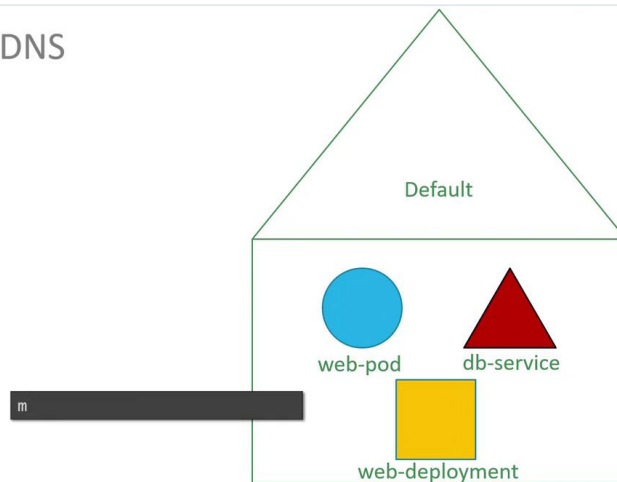
Namespaces allow you to set distinct policies and resource limits for different environments. This isolation prevents one namespace from interfering with another. For instance, you can apply separate resource quotas for CPU, memory, and the total number of pods to ensure fair usage across environments.

Namespace – Resource Limits



Within a single namespace, resources can refer to each other directly via their simple names. For example, a web application pod in the default namespace can access a database service simply by using its service name:

DNS



If the web app pod needs to communicate with a service located in a different namespace, you must use its fully qualified DNS name. For example, connecting to a database service named "db-service" in the "dev" namespace follows this format:

```
```SQL theme={null} mysql.connect("db-service.dev.svc.cluster.local")
```

Here, "svc" indicates the service subdomain, followed by the namespace ("dev") and the service name.

## ## Managing Namespaces with kubectl

Using `kubectl`, you can manage resources across different namespaces. Below are some common commands.

### ### Listing Pods in a Namespace

To list all pods in the default namespace:

```
```bash theme={null}
> kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
Pod-1         1/1     Running   0           3d
Pod-2         1/1     Running   0           3d
```

To list pods within the **kube-system** namespace:

```
```bash theme={null}
```

```
kubectl get pods --namespace=kube-system
NAME READY STATUS RESTARTS AGE
coredns-78cdf6894-92d52 1/1 Running 7 3d
coredns-78cdf6894-jx25g 1/1 Running 7 3d
etcd-master 1/1 Running 7 3d
kube-apiserver-master 1/1 Running 7 3d
kube-controller-manager-master 1/1 Running 7 3d
kube-flannel-ds-amd64-hz4cf 1/1 Running 14 3d
kube-proxy-4b8tn 1/1 Running 7 3d
kube-proxy-98db4 1/1 Running 7 3d
kube-proxy-jjrb5 1/1 Running 7 3d
kube-scheduler-master 1/1 Running 7 3d
```

### ### Creating Pods in Specific Namespaces

When creating a pod without specifying the namespace, it is placed in the default namespace.

```
```bash theme={null}
> kubectl create -f pod-definition.yml
pod/myapp-pod created
```

To create the same pod in the "dev" namespace, you can either include the namespace option:

```
```bash theme={null}
```

```
kubectl create -f pod-definition.yml --namespace=dev pod/myapp-pod
created
```

Or define the namespace within the pod definition file:

#### Without Namespace Specification

```
```yaml theme={null}
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  labels:
    app: myapp
    type: front-end
spec:
  containers:
    - name: nginx-container
      image: nginx
```

With Namespace Specification

```
```yaml theme={null} apiVersion: v1 kind: Pod metadata: name: myapp-pod
namespace: dev labels: app: myapp type: front-end spec: containers: - name:
nginx-container image: nginx
```

### Creating a Namespace

You can create a namespace using a YAML file. For example, create a file named `namespace-dev.yml`.

```
```yaml theme={null}
apiVersion: v1
kind: Namespace
metadata:
  name: dev
```

Then run:

```
```bash theme={null}
```

```
kubectl create -f namespace-dev.yml namespace/dev created
```

Alternatively, create a namespace directly through the command line:

```
```bash theme={null}
> kubectl create namespace dev
```

Setting the Default Namespace for Your Context

If you're working across multiple namespaces and want to avoid repeatedly specifying the namespace flag, you can set the default namespace for your current context:

```
```bash theme={null} kubectl config set-context $(kubectl config current-context) --namespace=dev
```

After setting this, running:

```
```bash theme={null}
> kubectl get pods
```

will automatically list pods in the "dev" namespace. To list pods from all namespaces, use:

```
```bash theme={null}
```

```
kubectl get pods --all-namespaces
```

<Callout icon="lightbulb" color="#1CB2FE">

Contexts are used to manage multiple clusters and user environments within a single configuration file.

## Controlling Resource Usage with ResourceQuotas

To ensure that no single namespace overconsumes cluster resources, Kubernetes allows you to

```
```yaml theme={null}
apiVersion: v1
kind: ResourceQuota
metadata:
  name: compute-quota
  namespace: dev
spec:
  hard:
    pods: "10"
    requests.cpu: "4"
    requests.memory: 5Gi
    limits.cpu: "10"
    limits.memory: 10Gi
```

Apply this configuration with:

```
```bash theme={null}
```

```
kubectl create -f compute-quota.yaml ```
```

This configuration guarantees that the "dev" namespace does not exceed the specified resource limits.

## Conclusion

Namespaces are a fundamental component in Kubernetes, enabling you to segment and manage resources effectively. Whether you're isolating system

components or separating development and production environments, using namespaces along with appropriate policies and resource quotas leads to a more efficient and organized cluster management.

Practice these concepts and explore additional Kubernetes functionalities to deepen your understanding. Happy clustering!

For further reading, check out these resources:

- [Kubernetes Documentation](#)
- [Kubernetes Basics](#)

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Solution Namespaces optional

This article provides a lab walkthrough on managing namespaces, pods, and services in Kubernetes.

This article provides a comprehensive lab walkthrough on namespaces in Kubernetes, addressing questions related to namespaces, pods, and services. Follow the steps below to learn how to manage namespaces effectively.

## Identifying the Number of Namespaces

To determine the number of namespaces in your cluster, you can run either of the following commands:

```
``bash theme={null} kubectl get namespaces
```

or its shorthand:

```
``bash theme={null}
kubectl get ns
```

The sample output below shows a list of namespaces:

```
``bash theme={null} controlplane ~ × kubectl get namespaces NAME STATUS
AGE default Active 6m50s kube-system Active 6m49s kube-public Active
6m49s kube-node-lease Active 6m49s finance Active 27s marketing Active 27s
dev Active 27s prod Active 27s manufacturing Active 27s research Active 27s
```

And using the shorthand command:

```
```bash theme={null}
controlplane ~ × kubectl get ns
NAME                STATUS    AGE
default             Active   6m55s
kube-system         Active   6m54s
kube-public         Active   6m54s
kube-node-lease     Active   6m54s
finance             Active   32s
marketing           Active   32s
dev                 Active   32s
prod                Active   32s
manufacturing       Active   32s
research            Active   32s
```

In this example, there are a total of 10 namespaces.

Counting Pods in the Research Namespace

To determine the number of pods running in the research namespace, execute the following command:

```
```bash theme={null} kubectl get pods --namespace=research
```

The output might look like this:

```
```bash theme={null}
controlplane ~ → kubectl get pods --namespace=research
NAME      READY   STATUS    RESTARTS   AGE
dna-2     0/1     CrashLoopBackOff   3 (14s ago)    76s
dna-1     0/1     CrashLoopBackOff   3 (16s ago)    76s
```

Alternatively, you can use the shorthand option:

```
```bash theme={null} kubectl get pods -n research
```

Both commands confirm that there are 2 pods in the research namespace.

\*\*\*

### ## Creating a Pod in the Finance Namespace

Before creating a pod in the finance namespace, ensure that the namespace exists by running

```
```bash theme={null}
kubectl get namespaces
```

You should see output similar to:

```
```bash theme={null} controlplane ~ » kubectl get namespaces NAME STATUS
AGE default Active 6m50s kube-system Active 6m49s kube-public Active
6m49s kube-node-lease Active 6m49s finance Active 32s marketing Active 32s
dev Active 32s prod Active 32s manufacturing Active 32s research Active 32s
```

Next, create a Redis pod named "redis" in the finance namespace with the following command:

```
```bash theme={null}
kubectl run redis --image=redis -n finance
```

After successfully executing this command, you will see a confirmation message:

```
```bash theme={null} pod/redis created
```

You can verify the pod creation by listing the pods in the finance namespace:

```
```bash theme={null}
kubectl get pods -n finance
```

Sample output:

```
```bash theme={null} NAME READY STATUS RESTARTS AGE payroll 1/1
Running 0 2m20s redis 0/1 ContainerCreating 0 8s
```

```

```

## ## Locating the Blue Pod's Namespace

To find out which namespace the blue pod is running in, you have two options:

1. Manually inspect each namespace using:

```
```bash theme={null}
kubectl get pods -n <namespace>
```
```

2. List all pods across every namespace with:

```
```bash theme={null}
kubectl get pods --all-namespaces
```
```

The sample output below illustrates pods across namespaces:

```
```bash theme={null}
```

NAMESPACE	NAME	READY	STATUS
kube-system	local-path-provisioner-6c796847f7-j5sqx	1/1	Running
kube-system	coredns-57889956d-bkj56	1/1	Running
kube-system	helm-install-traefik-crd-d2fg7	0/1	Completed
kube-system	metrics-server-7cd5fcbb67-kdgzb	1/1	Running
kube-system	helm-install-traefik-c987s	2/2	Running
kube-system	svclb-traefik-jptsh	1/1	Running
marketing	redis-db	1/1	Running
dev	redis-db	1/1	Running
manufacturing	red-app	1/1	Running
finance	payroll	1/1	Running
marketing	blue	1/1	CrashLoopBackOff
research	dna-1	0/1	CrashLoopBackOff
finance	redis	1/1	Running

From the output, it is clear that the blue pod is located in the **marketing** namespace.

Accessing the Database Service from the Blue Application

The blue application, a simple web application for connectivity tests, interacts with a database service. Depending on where the DB service is running, the access method varies.

Accessing the DB Service in the Same Namespace (Marketing)

In the marketing namespace, the blue application and its associated DB service coexist. To check the pods in this namespace, run:

```
```bash theme={null} kubectl get pods -n marketing
```

Expected output:

```
```bash theme={null}
NAME      READY   STATUS    RESTARTS   AGE
redis-db   1/1     Running   0           4m14s
blue       1/1     Running   0           4m14s
```

Next, list the services in the marketing namespace:

```
```bash theme={null} kubectl get svc -n marketing
```

This yields an output similar to:

```
```bash theme={null}
NAME            TYPE        CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
blue-service    NodePort    10.43.82.162    <none>           8080:30082/TCP   4m27s
db-service      NodePort    10.43.134.33    <none>           6379:30758/TCP   4m27s
```

Since both the blue application and the DB service are in the same namespace, the application can reach the DB service simply by using its DNS name, which is:

db-service

and connecting via port 6379.

Accessing the DB Service in the dev Namespace

Another DB service is running in the dev namespace. To view it, execute:

```
```bash theme={null} kubectl get svc -n dev
```

The output might be as follows:

```
```bash theme={null}
NAME            TYPE        CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
db-service      ClusterIP   10.43.252.9     <none>           6379/TCP         5m22s
```

To access the DB service in the dev namespace from the blue application located in the marketing namespace, you must use the fully qualified DNS name. The DNS naming convention is:

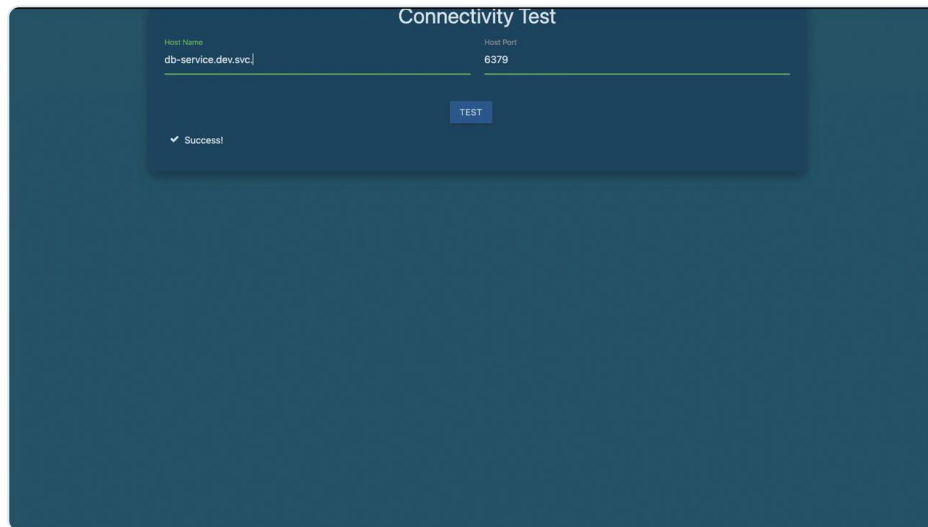
\\.svc.cluster.local

In this case, the DNS name is:

db-service.dev.svc.cluster.local

Using this fully qualified domain name ensures traffic is correctly directed to the DB service in the dev namespace on port 6379.

When performing connectivity tests from the blue application, the console output should verify that the connection to the DB service is successfully established.



This concludes the lab solution for namespaces. Each step demonstrates how to manage and interact with namespaces, pods, and services in a Kubernetes cluster.

For more details on Kubernetes concepts, visit the [Kubernetes Documentation](#).

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Imperative vs Declarative

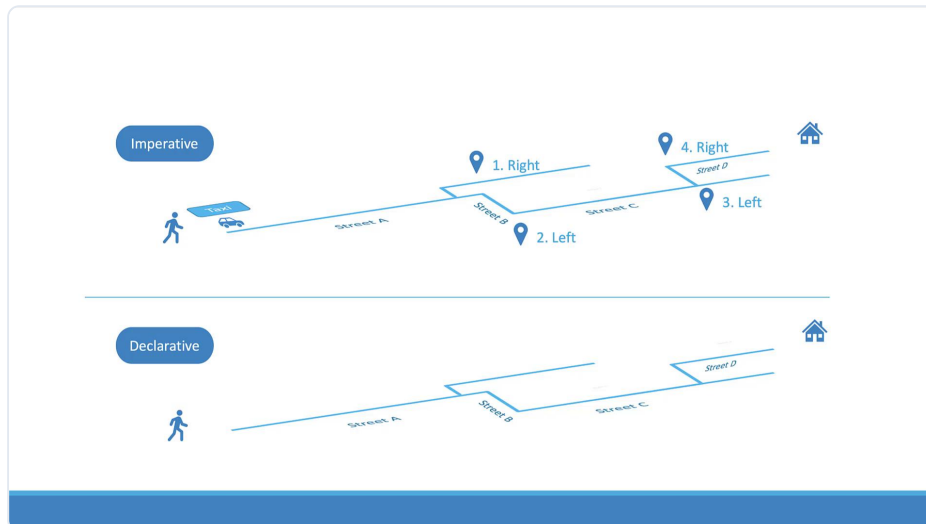
This article explores imperative and declarative approaches to managing Kubernetes objects, highlighting their benefits, drawbacks, and ideal use cases.

In this lesson, we explore the two core approaches used to manage Kubernetes objects: imperative and declarative. By understanding these methods and their respective benefits and drawbacks, you'll be better equipped to manage your clusters and prepare for Kubernetes certification exams.

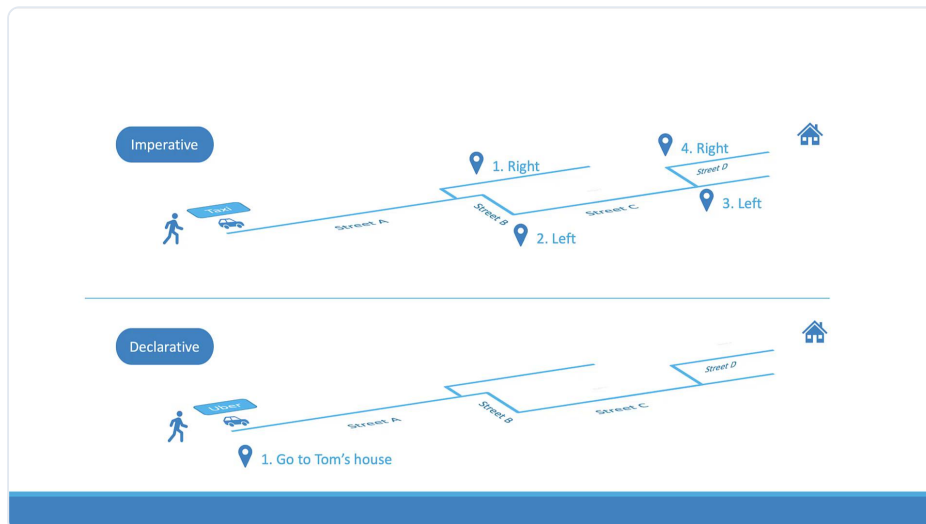
So far, we have covered creating and managing Kubernetes objects either by directly issuing commands or by using configuration files. These approaches fall into two broad categories in the infrastructure-as-code domain: imperative and declarative.

An Analogy

Imagine visiting a friend's house. In the past, you might have taken a taxi and given the driver precise, step-by-step directions—for example, "Take a right onto Street B, then left onto Street C, another left onto Street D, and finally stop at the house." This sequence of detailed instructions illustrates the imperative approach.



Today, using an app like Uber, you simply enter your final destination. This is akin to the declarative approach where you specify the desired outcome —"Drive to Tom's house"—and the system figures out the best route.



Imperative vs Declarative: Kubernetes Perspective

Imperative Approach

The imperative approach in Kubernetes involves executing specific commands to create, update, or delete objects. This method instructs Kubernetes on both what needs to be done and how it should be done. For example, you might run these commands:

```
``bash theme={null} kubectl run --image=nginx nginx kubectl create deployment --image=nginx nginx kubectl expose deployment nginx --port 80 kubectl edit deployment nginx kubectl scale deployment nginx --replicas=5 kubectl set image deployment nginx nginx=nginx:1.18 kubectl create -f nginx.yaml kubectl replace -f nginx.yaml kubectl delete -f nginx.yaml
```

While the imperative approach is effective for quick tasks, it comes with some limitations:

- * If a command partially executes, running it again may require extra checks (e.g., verify)
- * Updating resources—such as changing the image version—demands explicit re-execution with
- * Commands executed interactively are often not persisted, making it challenging for teams

In Kubernetes, imperative commands like `kubectl run`, `kubectl create deployment`, `kubectl`

<Frame>

!The image compares imperative and declarative approaches in Infrastructure as Code, del

</Frame>

Declarative Approach

The declarative approach enables you to specify the desired state of your infrastructure th

```
```yaml theme={null}
nginx.yaml
apiVersion: v1
kind: Pod
metadata:
 name: myapp-pod
 labels:
 app: myapp
 type: front-end
spec:
 containers:
 - name: nginx-container
 image: nginx
```

Applying the configuration is as simple as running:

```
```bash theme={null} kubectl apply -f nginx.yaml
```

Kubernetes will create or update the object automatically to match the state described in y

```
```bash theme={null}
kubectl apply -f nginx.yaml
```

This method ensures that your configuration files remain the single source of truth, which is especially valuable in team environments where version-controlled definitions are critical.

## Imperative vs Declarative Update Dilemma

Sometimes, you might modify a live object using the `kubectl edit` command. This command opens a YAML representation of the current state, including additional fields like status, which are absent from your original configuration file. For instance:

1. Initially, you create the object using your YAML file:

```
bash theme={null} kubectl create -f nginx.yaml
```

1. Later, you edit the deployment:

```
bash theme={null} kubectl edit deployment nginx
```

1. The live object now contains extra status fields. If you later apply the original `nginx.yaml` (perhaps with updates), your live edits might be overwritten.

Always update your local configuration files and use commands like `kubectl replace -f nginx.yaml` to ensure that your changes are consistently tracked and version-controlled.

A typical workflow in a team environment is as follows:

- Create the object:

```
bash theme={null} kubectl create -f nginx.yaml
```

- Modify the local file to implement changes (e.g., update the image version).
- Update the live object with:

```
bash theme={null} kubectl replace -f nginx.yaml
```

This practice reinforces a version-controlled process and promotes collaboration.

## Choosing the Right Approach

Approach	Ideal Use Case	Example Commands
Imperative	Quick, one-off tasks such as creating a pod or deployment.	<pre>kubectl run -- image=nginx nginx kubectl create deployment --image=nginx nginx kubectl expose deployment nginx --port 80</pre>
Declarative	Long-term management with version-controlled infrastructure.	<pre>kubectl apply -f nginx.yaml kubectl apply -f /path/ to/config-files</pre>

- **Imperative Approach:** Use this method for rapid execution when you need to quickly create or modify Kubernetes objects, particularly during certification exams.

- **Declarative Approach:** This approach is recommended for complex, long-term management scenarios. It enables a systematic management of configurations via YAML files, ensuring every change is recorded and version-controlled.

## Exam Tips

---

When preparing for Kubernetes certification exams, consider the following strategies:

- Use imperative commands for speed when creating simple objects like pods or deployments.
- For modifications or more intricate configurations, adopt the declarative approach by updating configuration files and applying changes using `kubectl apply` or `kubectl replace`.
- Always maintain your YAML files in version control to safeguard against unintentional overwrites.

For more detailed guidance on managing a Kubernetes cluster, check the [official Kubernetes documentation](#) and experiment with both approaches in your lab exercises.

Happy learning, and see you in the next lesson!

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

# Kubectl Apply Command

This article explores the kubectl apply command, its internal workings, and how it manages Kubernetes object configurations declaratively.

In this article, we explore how the kubectl apply command works and what happens internally during its execution. Using kubectl apply for declarative management of Kubernetes objects is common practice, and here we dive into details such as configuration comparisons and update processing.

## Basic Example

Consider the following local YAML configuration file (nginx.yaml) that defines a Pod:

```
```yaml theme={null} apiVersion: v1 kind: Pod metadata: name: myapp-pod labels: app: myapp type: front-end-service spec: containers: - name: nginx-container image: nginx:1.18
```

Apply this configuration with:

```
```bash theme={null}
kubectl apply -f nginx.yaml
```

You can also apply all configuration files within a directory:

```
```bash theme={null} kubectl apply -f /path/to/config-files
```

How kubectl apply Works Internally

When you run the kubectl apply command, it compares three sources:

1. The local configuration file (e.g., nginx.yaml).
2. The live object configuration stored on the Kubernetes cluster.
3. The last applied configuration stored as an annotation on the live object.

If the object does not exist, Kubernetes creates it based on your local configuration. During

<Callout icon="lightbulb" color="#1CB2FE">

When the local configuration is changed (for example, updating the image version), kubectl
</Callout>

For instance, if you update the image version from 1.18 to 1.19 in your local file:

```
```yaml theme={null}
apiVersion: v1
kind: Pod
metadata:
 name: myapp-pod
 labels:
 app: myapp
 type: front-end-service
spec:
 containers:
 - name: nginx-container
 image: nginx:1.19
```

and run:

```
```bash theme={null} kubectl apply -f nginx.yaml
```

kubectl compares the three configurations. If differences are detected—such as the updated

Managing Removed Fields

The last applied configuration annotation is crucial when fields are removed from your local

```
```yaml theme={null}
apiVersion: v1
kind: Pod
metadata:
 name: myapp-pod
 labels:
 app: myapp
spec:
 containers:
 - name: nginx-container
 image: nginx:1.19
```

and run:

```
```bash theme={null} kubectl apply -f nginx.yaml
```

```
kubectl notices that the "type" label, which existed in the last applied configuration, is
## Last Applied Configuration Annotation

When kubectl apply is executed for the first time, the YAML configuration is converted to
```yaml
theme: {null}
apiVersion: v1
kind: Pod
metadata:
 name: myapp-pod
 annotations:
 kubectl.kubernetes.io/last-applied-configuration: '{"apiVersion": "v1", "kind": "Pod", "met
 labels:
 app: myapp
 type: front-end-service
spec:
 containers:
 - name: nginx-container
 image: nginx:1.18
status:
 conditions:
 - lastProbeTime: null
```

This annotation is the key to performing a three-way merge in future apply operations. The process compares:

- The local file.
- The live object configuration.
- The last applied configuration.

This comparison ensures that Kubernetes makes precise updates. It's important to note that mixing imperative commands (like `kubectl create` or `kubectl replace`) with declarative ones can lead to inconsistencies, as only `kubectl apply` stores the last applied configuration.

## Actionable Summary

Step	Description	Example Command
Initial Creation	Creates the object and stores the configuration as an annotation	kubectl apply -f nginx.yaml
Update Configuration	Modifies the object by comparing local, live, and last applied configuration	kubectl apply -f nginx.yaml
Remove a Field	Deletes a field from live configuration when it is removed locally	kubectl apply -f nginx.yaml

Avoid mixing imperative commands with declarative approaches. Imperative actions like `kubectl create` or `kubectl replace` will not record the last applied configuration and may lead to inconsistencies when using `kubectl apply`.

## Conclusion

---

Understanding how `kubectl apply` processes local configurations, the live state of Kubernetes objects, and the last applied configuration annotation is crucial for managing resources declaratively. For more comprehensive information, please refer to the [Kubernetes documentation](#).

Thank you for reading this guide on the `kubectl apply` command. Happy deploying!

## Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

## A Quick Reminder

This article emphasizes the importance of following labs and video lectures for effective learning in the course.

In this article, we emphasize the importance of following the provided labs and video lectures in this course. While it might be tempting to set up your own local environment right away, our structured approach is designed to guide you step-by-step through the core concepts without distractions.

Focus on the course content as provided to maximize your learning efficiency and avoid unnecessary detours.

Following the labs and lectures in the intended order ensures a seamless learning experience, allowing you to build foundational skills before moving on to more advanced topics such as configuring your local environment. This method helps you optimize your study time and develop the necessary skills more effectively.

Below is an example output from the command that lists all Docker images available on your system:

```
```bash theme={null} $ docker images REPOSITORY TAG SIZE redis latest 105MB ubuntu latest 72.7MB mysql latest 556MB nginx latest 22.6MB alpine alpine 5.6MB nginx latest 133MB postgres latest 314MB kodekloud/simple-webapp-mysql latest 96.6MB kodekloud/simple-webapp latest 84.8MB
```

This output showcases the various Docker images installed, which will become increasingly relevant as you progress through the course.

For now, please proceed with the lab that involves running the following command:

```
```bash theme={null}
$ docker run redis
```

This lab is specifically designed to build your foundational Docker skills. Additional resources and detailed guidance—such as instructions on setting up your local environment—will be provided later in the course.

Thank you for choosing our course. We are excited to support you on your journey to mastering Docker and related technologies.